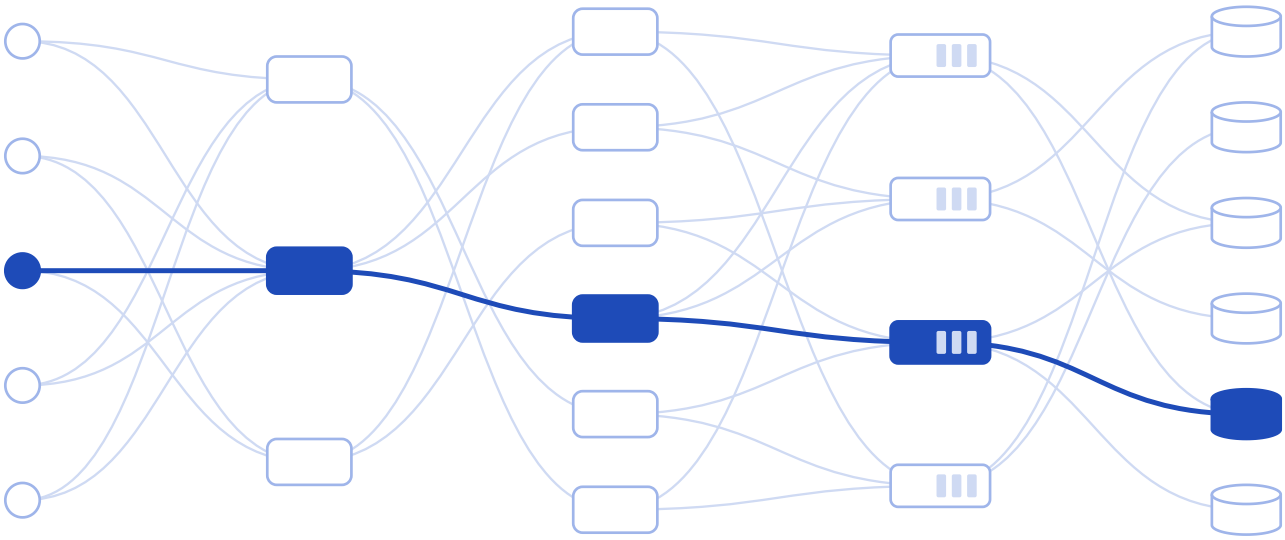
SYSTEM DESIGN FIELD GUIDE

Scaling & Reliability Patterns

Twenty-one patterns that show up in every serious production system — the problem each one solves, what it quietly costs you, and how to reason about it out loud in a senior design interview.



PART I
Serving reads fast

PART II
Scaling writes & data

PART III
Asynchronous work & decoupling

PART IV
Designing for failure

PART V
Real-time delivery

PART VI
Putting it together

Edition 1.0 · October 2026

For senior engineers, staff engineers & interview preparation

CONTENTS

What's inside

• **Before you start**

How to use this guide 3
The decision map 4

I Serving reads fast 5

01 Indexing make each query cheaper 6
02 Caching avoid repeated work 8
03 Read replicas add read capacity 10

II Scaling writes & data 12

04 Batching amortize per-request overhead 13
05 Partitioning divide a huge table 14
06 Sharding spread data across nodes 16
07 Consistent hashing move little when nodes change 18

III Asynchronous work & decoupling 20

08 Async processing shorten the request path 21
09 Message queues buffer and distribute work 23
10 Pub/Sub fan one event out 25
11 Event-driven architecture couple services through facts 26
12 Job queues durable deferred work 28
13 Background workers execute it safely 30
14 Workflow engines durable multi-step processes 32

IV Designing for failure 34

15 Timeouts bound the wait 35
16 Retries with backoff recover from blips 37
17 Circuit breakers stop hammering the sick 39
18 Idempotency make repeats harmless 40

V Real-time delivery 42

19 Long polling near-real-time over plain HTTP 43
20 Server-Sent Events one-way streaming 45
21 WebSockets two-way, persistent 47

VI Putting it together 49

From symptom to pattern 50
The four questions 51
Worked example: a flash-sale checkout 52
Final cheat sheet 54

BEFORE YOU START

How to use this guide

Start from the bottleneck, not the technology.

Every pattern in this guide buys you something — read capacity, write throughput, resilience, responsiveness — and pays for it with something else: freshness, simplicity, ordering, consistency, or a new system someone has to run at 3 a.m. Nothing here is free, and nothing here is a default. Kafka is not an architecture; it is a trade.

So the guide is organized the way problems actually arrive in production. Part I deals with reads that are too slow, Part II with writes and data that outgrow a machine, Part III with work that does not belong on the request path, Part IV with dependencies that fail, and Part V with clients that need updates the moment they happen. Part VI pulls it together into a repeatable way of reasoning in an interview.

Within each part, patterns are ordered roughly from cheapest to most expensive to adopt. That ordering is the first lesson: a senior engineer reaches for the composite index before the read replica, and for the read replica before the shard.

The strongest signal you can send in a design review or interview is naming the cost of a pattern before anyone asks — "this cache means profile edits can take up to sixty seconds to show" — and then saying why that cost is acceptable here.

Anatomy of a pattern

Each pattern opens with a three-part spec — **Solves**, **Costs** and **Consistency** — followed by a production scenario, the mechanics, a diagram, and four colour-coded blocks you will find on every pattern.

⚠ THE PROBLEM
A realistic production situation where this pattern earns its place. Read it first.

✓ REACH FOR IT WHEN
The signals that justify the pattern — symptoms, workloads, constraints.

! WATCH OUT FOR
Failure modes and costs that bite in production, usually months later.

💬 INTERVIEW CUE
How a strong candidate frames it out loud — and the follow-up to expect.

Diagram language

Every diagram uses the same shapes and line styles, so you can read a figure before reading its caption.

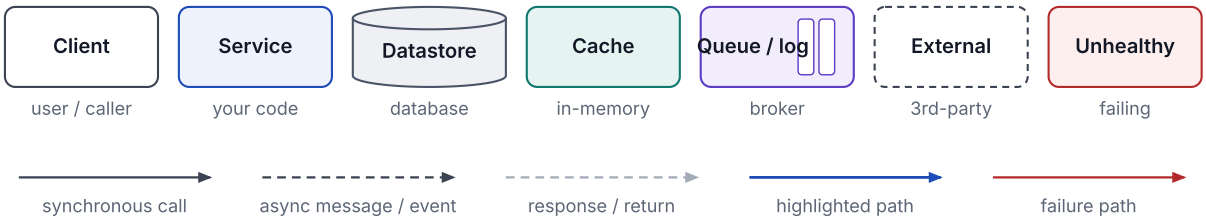


Figure 0.1 Node and edge conventions. Dashed outlines mean "outside your control"; red always means a failure path or an unhealthy component.

BEFORE YOU START

The decision map

Name the pressure first. The pattern follows; the price comes with it.

PRESSURE ON THE SYSTEM	PATTERNS THAT RELIEVE IT	WHAT YOU PAY WITH	SEE
Reads are slow or the database is read-bound	Indexing · caching · read replicas	Write overhead per index, stale data, replication lag	01–03
Writes are slow or the primary is write-bound	Batching · async processing · sharding	Added latency, ordering questions, distributed complexity	04 · 06 · 08
The dataset outgrows one table or node	Partitioning · sharding · consistent hashing	Hotspots, rebalancing, cross-partition queries	05–07
Services are tightly coupled	Queues · events · pub/sub	Duplicates, ordering, eventual consistency, harder debugging	09–11
Work is slow, long-running or deferred	Job queues · workers · workflow engines	Retries, duplicate execution, state to manage	12–14
Dependencies fail or slow down	Timeouts · retries + backoff · circuit breakers · idempotency	Tuning effort, retry storms if done wrong, unknown outcomes	15–18
Clients need live updates	Long polling · SSE · WebSockets	Long-lived connections, fan-out, reconnect behaviour	19–21

Three rules that hold across every part

MEASURE FIRST

Know the read/write ratio, peak rate, payload size and latency target before choosing. "It might get big" is not a requirement.

CHEAPEST FIX FIRST

A missing index, an N+1 query or a chatty client is a cheaper fix than a new subsystem. Complexity is a cost you pay forever.

EVERY ARROW CAN FAIL

Any network hop can time out, run twice or arrive late. Design the failure behaviour in the same pass as the happy path.

HOW TO USE THIS PAGE IN AN INTERVIEW

Restate the requirement as a pressure ("this is read-heavy, 100:1, and profile data can be a minute stale"), pick the cheapest pattern that relieves that pressure, state its cost in one sentence, then ask the failure question: *what happens when this piece is slow, down, or runs twice?*

Serving reads fast

Most production systems read far more than they write — 10:1 is common, 100:1 is not unusual. Read scaling is therefore where the cheapest wins live, and where teams most often reach for heavy machinery they don't yet need.

THE QUESTION THIS PART ANSWERS

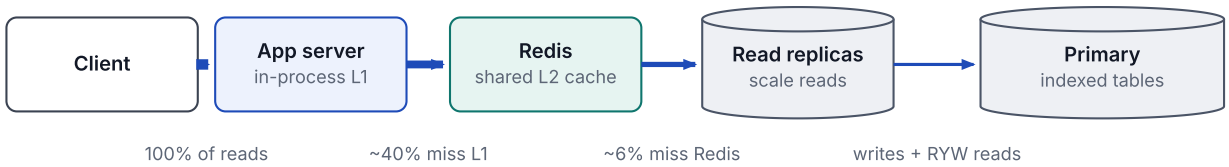
Is the database slow because each query is expensive, because the same query runs over and over, or because there is simply too much read traffic for one machine?

01 **Indexing**
Make each individual query cheaper.

02 **Caching**
Stop running the same query repeatedly.

03 **Read replicas**
Add machines to absorb aggregate read load.

→ **Order matters**
Each step is more expensive to run than the last.



Illustrative numbers. Every layer you add in front of the database trades freshness for capacity.

Figure I The layered read path. Each layer absorbs a share of traffic before it reaches the next; the primary should only see writes and reads that must be fresh.

TECHNIQUE	WHAT IT SCALES	CONSISTENCY COST	OPERATIONAL COST	TRY IT WHEN
Indexing	Cost of a single query	None	Low — slower writes, more storage	A few query shapes dominate load
Caching	Repeated identical reads	Stale up to TTL	Medium — invalidation, new failure mode	Access is skewed toward hot keys
Read replicas	Total read throughput	Replication lag	Medium — routing, failover	Reads are broad, not just a hot set

01 Indexing

Make each query cheaper before you make the database bigger.

SOLVES	COSTS	CONSISTENCY
Slow queries caused by scanning far more rows than they return	Extra work on every insert, update and delete; storage per index	None — indexes update in the same transaction as the row

⚠ THE PROBLEM

An `orders` table has grown to 80 million rows. The "my orders" page runs `WHERE customer_id = ? ORDER BY created_at DESC LIMIT 20`, and its p99 has crept to 2.4 seconds with database CPU pinned at 90%. The team is already pricing read replicas. The query plan shows a sequential scan: to return 20 rows, Postgres reads the whole table.

How it works

An index is a second, sorted copy of a few columns plus a pointer back to each row. Most relational indexes are **B-trees**: a shallow, wide tree in which finding a key takes a handful of page reads regardless of table size — logarithmic, not linear. Add a composite index on `(customer_id, created_at)` and that query becomes a range scan that touches about twenty rows. Latency drops from seconds to single-digit milliseconds, and the replica conversation is over.

Column order is the whole game in a composite index. The tree is sorted by the first column, then the second within it, so it serves any query that uses a *leftmost prefix*. Put equality filters first and the range or sort column last. If the index also contains every column the query selects, it becomes a **covering index** and the database never touches the table at all.

The price is paid on writes. Every insert updates every index on the table, so a table with eight indexes does roughly nine writes per insert. Indexes also take memory, and a working set that no longer fits in RAM turns fast lookups into disk reads. Low-selectivity columns — a `status` with four values — rarely help; the planner will scan anyway.

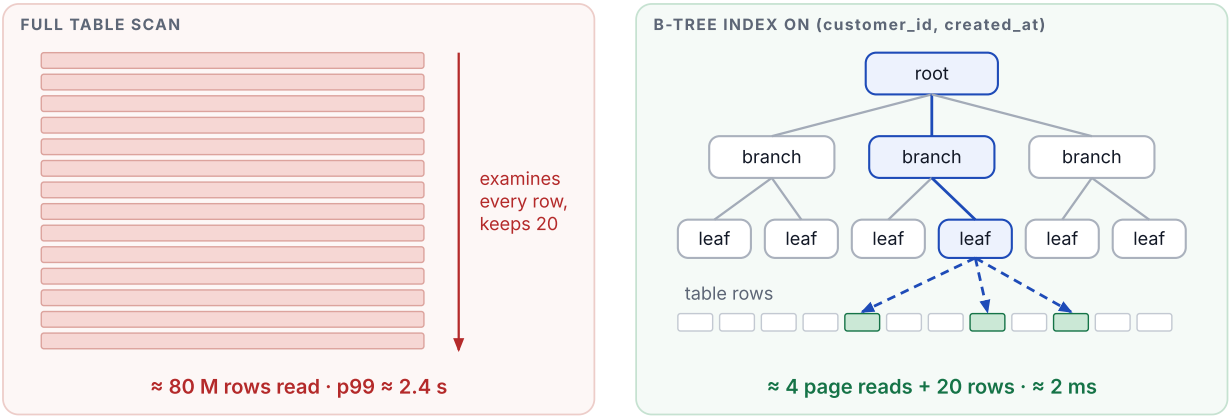


Figure 1.1 The same query with and without a suitable index. The B-tree walks one path from root to leaf, then fetches only the matching rows.

WHICH QUERIES CAN USE AN INDEX ON (CUSTOMER_ID, CREATED_AT)?

QUERY SHAPE	USES INDEX?	WHY
`WHERE customer_id = ?`	Yes	Leftmost column
`WHERE customer_id = ? ORDER BY created_at DESC`	Yes, no sort	Rows already ordered within each customer
`WHERE created_at > now() - interval '1 day'`	No	Skips the leading column; needs its own index
`WHERE lower(email) = ?` on a plain `email` index	No	A function hides the column; use an expression index

REACH FOR IT WHEN

- `EXPLAIN ANALYZE` shows scans on selective filters.
- A few query shapes account for most database time.
- Before any replica, cache or shard discussion.

WATCH OUT FOR

- Indexing every column — unused indexes still tax every write. Audit usage stats.
- Building an index on a large live table can lock writes; use `CREATE INDEX CONCURRENTLY` or online DDL.
- Indexes that outgrow memory.

INTERVIEW CUE

"Before scaling the database, I'd look at the query plan for the top queries. A surprising number of 'we need sharding' conversations end with a missing composite index."

02 Caching

Serve repeated reads from memory — and decide up front how stale is acceptable.

SOLVES	COSTS	CONSISTENCY
Repeated, expensive reads of the same data	Invalidation logic, memory, and a new component that can fail	Readers may see data up to one TTL old, or until invalidated

⚠ THE PROBLEM

A product detail page serves 40,000 requests per second. Rendering it takes six queries and a call to the pricing service. Traffic is heavily skewed — 2% of products receive 95% of views — so the database spends most of its day answering the same questions with the same answers.

How it works

Put a fast key-value store such as Redis between the application and the source of truth. The default strategy is **cache-aside**: read from the cache; on a miss, load from the database and store the result with a TTL. On writes, update the database and then *delete* the cache key rather than overwriting it — deleting avoids two concurrent writers leaving the older value in the cache.

Even delete-on-write has a race: a slow reader can load the old row, the writer commits and deletes, and the reader then writes the stale value back. That is why every cached entry still needs a TTL — it bounds how long any bug or race can serve wrong data. Add a little random jitter to TTLs so that keys written together don't all expire together.

The dangerous moment is a miss under load. When a hot key expires, thousands of concurrent requests miss at once and all hit the database — a **stampede**. Defend with **request coalescing** (one loader per key, everyone else waits), serving stale-while-revalidate, or refreshing hot keys early. And plan for the cache being empty: after a restart or failover, can the database survive a cold start?

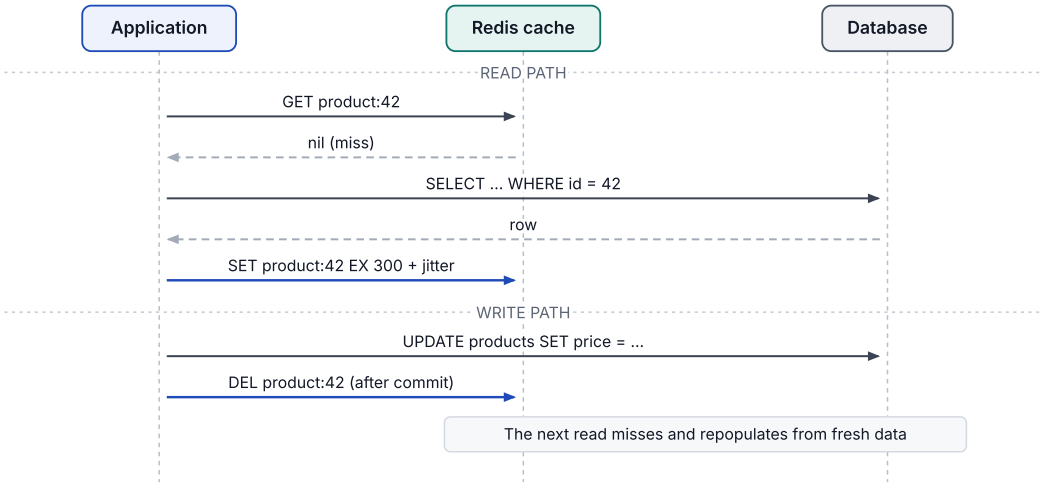


Figure 2.1 Cache-aside. The application owns both paths; the cache never talks to the database. Delete after the database commit, never before.

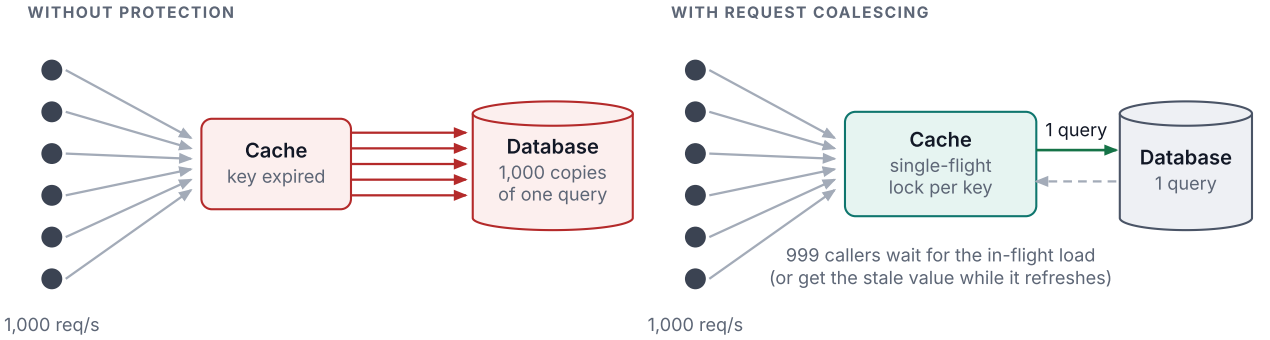


Figure 2.2 A hot key expiring under load. Coalescing turns a thousand identical database queries into one.

WRITE STRATEGIES AT A GLANCE

STRATEGY	HOW WRITES FLOW	CHOOSE IT WHEN
Cache-aside	App writes the database, then deletes the key	The default; simple and resilient to cache loss
Write-through	App writes cache and database together, synchronously	Reads must be fresh right after writes; write latency is acceptable
Write-back	Write to cache, flush to the database later	Very high write rates, and losing recent writes is tolerable
Refresh-ahead	Reload hot keys before they expire	Predictable hot set where a miss is very expensive

REACH FOR IT WHEN

- Reads far outnumber writes and access is skewed.
- Results are expensive to compute or fetch.
- The business can state a tolerable staleness.

WATCH OUT FOR

- Cache keys that omit a dimension — tenant, locale, permissions — and leak one user's data to another.
- Hot keys that saturate a single cache node; add a small in-process cache.
- The cache quietly becoming the source of truth.

INTERVIEW CUE

"First question: how stale can this data be? Second: what happens to the database when the cache is empty — after a restart, or when a hot key expires?"

03 Read Replicas

Spread reads across copies of the database — then design around the lag.

SOLVES	COSTS	CONSISTENCY
The read-throughput ceiling of a single database node	Replication lag, routing logic, more nodes to operate	Replicas trail the primary by milliseconds — sometimes seconds

⚠ THE PROBLEM

Checkout and the internal reporting dashboards share one Postgres primary. Reads are 90% of the load, the reads are varied enough that caching only helps a little, and every time finance runs month-end reports, checkout latency doubles. Writes, on their own, are comfortably within capacity.

How it works

The primary accepts all writes and streams its change log — the WAL in Postgres, the binlog in MySQL — to one or more replicas, which replay it. Read-only queries are routed to replicas by the application, a driver, or a proxy. Aggregate read capacity now grows with each replica, and heavy analytical queries can be isolated on a replica of their own where they cannot hurt checkout.

Replication is usually **asynchronous**: the primary confirms a commit without waiting for replicas. That keeps writes fast but means each replica is somewhat behind — normally milliseconds, but seconds under heavy write load or a long-running query on the replica. Synchronous or semi-synchronous replication waits for at least one replica, trading write latency for durability.

Lag creates the classic bug: **read-after-write**. A user saves a change, the next page load hits a replica that hasn't caught up, and they see the old value. Typical fixes: route a user's reads to the primary for a few seconds after they write; send consistency-critical reads to the primary always; or have the client carry its last write position and only use a replica that has replayed past it.

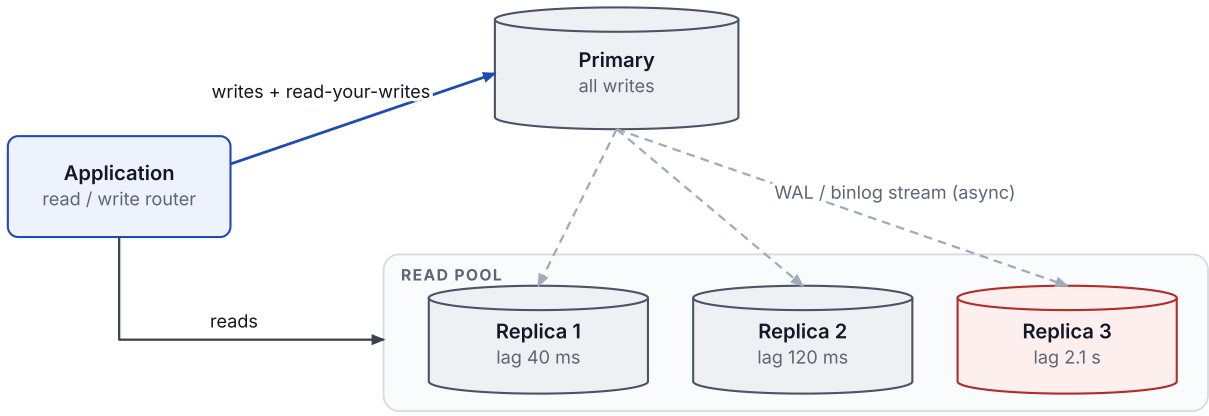


Figure 3.1 Writes go to the primary; reads spread across the pool. Lag varies per replica, and one lagging replica is enough to serve surprising answers.

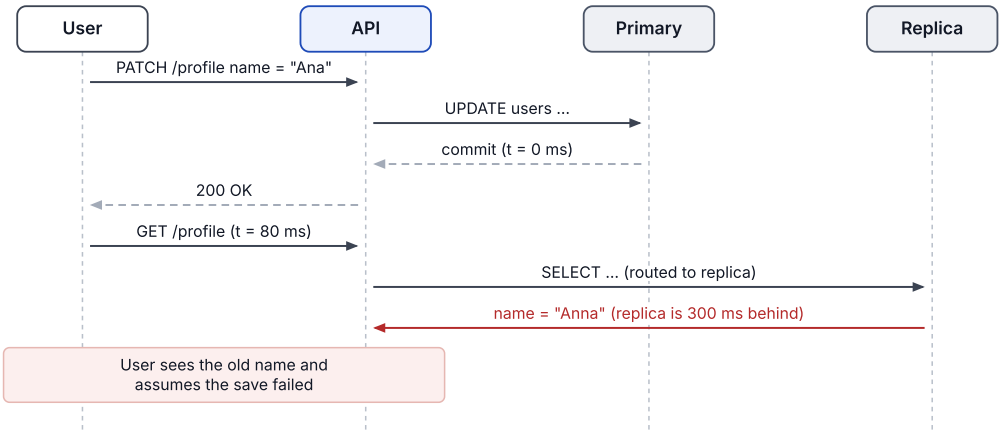


Figure 3.2 The read-after-write anomaly. Nothing is broken — the system is simply eventually consistent, and the UI did not account for it.

REACH FOR IT WHEN

- Reads dominate, are broad, and tolerate a little staleness.
- Analytics or reporting must be isolated from OLTP traffic.
- You also want a warm standby for failover.

WATCH OUT FOR

- Replicas don't help write bottlenecks — every replica replays every write.
- Async failover can lose writes the primary had already acknowledged.
- Lag spikes; alert on it, and stop routing to a replica that falls behind.

INTERVIEW CUE

"Replicas scale reads, not writes. I'd also say up front how we handle read-your-own-writes, because that's the bug users will actually notice."

PART II OF VI

II

Scaling writes & data

Reads can be copied; writes cannot. Every replica replays every write, so when the primary itself runs out of write throughput or disk, the only way forward is to make writes cheaper or to split the data so that no single machine owns all of it.

THE QUESTION THIS PART ANSWERS

Is the write path expensive because of per-operation overhead, because one table has become unmanageable, or because one machine has genuinely reached its limit?

04 **Batching**
Amortize the fixed cost of each write.

05 **Partitioning**
Split one huge table into manageable pieces.

06 **Sharding**
Spread those pieces across machines.

07 **Consistent hashing**
Decide which machine owns which key — cheaply.

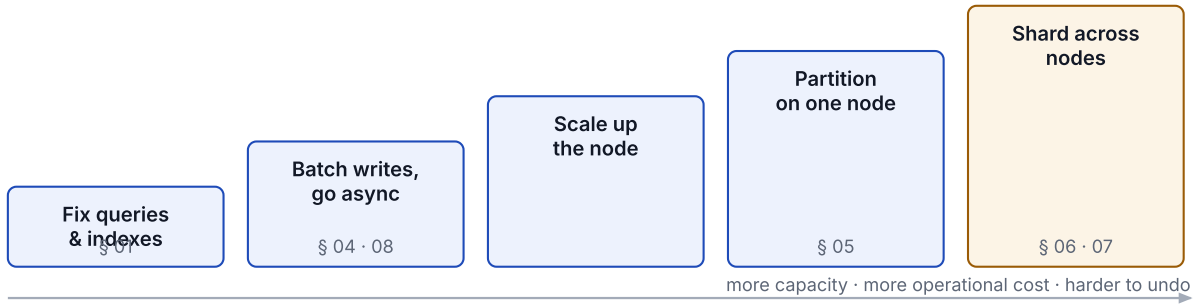


Figure II The escalation ladder. Each rung buys capacity and costs complexity; the top rung is the hardest to climb back down from.

TECHNIQUE	PRIMARY PURPOSE	WHERE DATA LIVES	MAIN TRADE-OFF
Read replicas	Scale reads	Full copy on every node	Writes still all hit one primary
Partitioning	Split data logically	One node, many physical pieces	Queries without the partition key touch every piece
Sharding	Distribute data and write load	Different subsets on different nodes	Cross-shard queries, transactions and rebalancing

04 Batching

Pay the fixed cost of a round trip once, not a thousand times.

SOLVES	COSTS	CONSISTENCY
Per-operation overhead: round trips, commits, fsyncs, request fees	Extra latency while a batch fills; larger units of failure	Unchanged — but partial failures become a normal case

⚠ THE PROBLEM

An IoT ingestion service writes each sensor reading as its own transaction — 20,000 inserts per second. Each commit forces a log flush to disk, and the database tops out around 5,000 commits per second long before CPU or disk bandwidth is exhausted. The work is tiny; the ceremony around it is the bottleneck.

How it works

Accumulate operations in a buffer and send them together: a multi-row `INSERT` or `COPY`, a bulk API call, a Kafka producer batch. Writing 500 rows per transaction turns 20,000 commits per second into 40, and throughput typically jumps by an order of magnitude on the same hardware.

Always bound a batch two ways — by **size** and by **time** — and flush on whichever comes first. The size cap protects memory and keeps requests under API limits; the time cap protects latency during quiet periods, so a lone event never waits forever for company. Kafka's `batch.size` and `linger.ms` are exactly this pair.

Batches fail differently. Many bulk APIs report per-item results — DynamoDB returns `UnprocessedItems` — so the caller must retry only what failed, idempotently. One malformed record shouldn't sink 499 good ones: isolate it, split the batch, and dead-letter the bad item. And remember an in-memory buffer dies with the process; acknowledge upstream only after the flush succeeds, or buffer in a durable log.

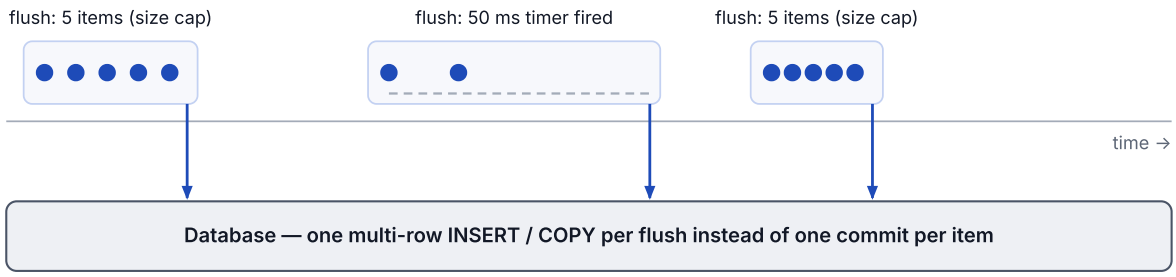


Figure 4.1 A batcher flushes on size during bursts and on a timer during quiet periods, bounding both memory and the latency any single item can experience.

✓ REACH FOR IT WHEN

- Many small, similar operations dominate cost.
- The downstream charges or rate-limits per request.
- Throughput matters more than per-item latency — ingestion, analytics, notifications.

! WATCH OUT FOR

- A latency floor equal to the flush interval.
- Unbounded buffers when the downstream slows — block or shed instead.
- Lost data on crash, and ordering across retried partial batches.

💬 INTERVIEW CUE

"If each write is expensive mostly because of per-request overhead, batching is cheaper than more hardware. I'd flush on size or time, whichever comes first, and retry failed items individually."

05 Partitioning

Divide one huge table into pieces the database can skip, drop and maintain on their own.

SOLVES	COSTS	CONSISTENCY
Tables too large to query, index and maintain efficiently	A partition key that is hard to change; slower queries that ignore it	Unchanged on a single node — still one database

⚠ THE PROBLEM

An `events` table holds four billion rows. Nearly every query asks about the last seven days, yet indexes cover all history and no longer fit in memory. Deleting data older than a year with `DELETE` runs for hours, generates enormous write-ahead log volume, and leaves the table bloated.

How it works

Declare the table as partitioned by a key, and the database stores each range or value set as a separate physical table behind one logical name. **Range** partitioning by time is the workhorse; **list** partitioning groups known values such as region or tenant tier; **hash** partitioning spreads rows evenly when there is no natural range.

The payoff is **partition pruning**: when a query filters on the partition key, the planner skips every partition that cannot match. Each partition also carries its own smaller indexes, so the hot month's index fits in memory even when the full history would not. Retention becomes a metadata operation — detach and drop last year's partition in milliseconds, with no vacuum debt.

Everything depends on the key matching how the data is accessed. Queries that don't filter on it must visit every partition, which can be slower than a single table. Uniqueness constraints usually have to include the partition key. And too many tiny partitions burden the planner — prefer tens to low thousands, not millions.

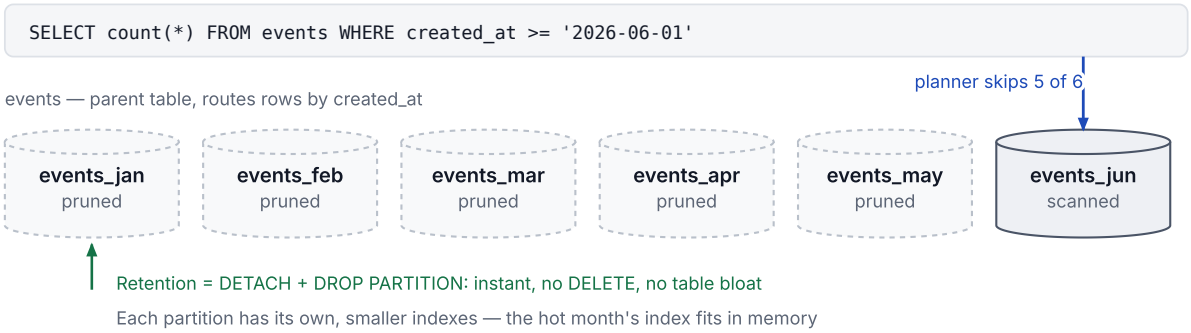


Figure 5.1 A monthly range-partitioned table. A query on recent data scans one partition; retention drops whole partitions instead of deleting rows.

CHOOSING A PARTITIONING SCHEME

SCHEME	BEST FOR	MAIN RISK
Range	Time series, logs, orders by date — anything with retention	All new writes land on the newest partition
List	Region, tenant tier, product line — a known set of values	Skew when one value dominates
Hash	Even spread when no natural range exists	Range queries visit every partition

Follow-ups to expect

Q How is partitioning different from sharding?

Partitioning splits a table into pieces inside one database; sharding puts the pieces on different servers. Partitioning keeps transactions, joins and a single failure domain.

Q What if most queries don't include the key?

Then the key is wrong for this workload. Pick one that matches access, or keep a small lookup structure for the other access path.

Q Why not just index created_at?

An index speeds lookups but not retention: deleting a billion rows still bloats the table, and the index grows with all of history. Partitions bound both.

Q How do you choose the granularity?

Size partitions so the hot ones and their indexes fit in memory and the total count stays manageable — daily at very high volume, monthly otherwise.

REACH FOR IT WHEN

- A table is very large and queries naturally target a slice.
- Data has a retention policy.
- Tenants or regions need isolation on one database.

WATCH OUT FOR

- Queries that omit the key and silently scan everything.
- Skewed partitions — one giant tenant.
- Forgetting to pre-create future partitions.

INTERVIEW CUE

"Partitioning is the broad idea of dividing data. Sharding is when those partitions live on different machines — and that's where the distributed-systems costs start."

06 Sharding

When one machine can't take the writes, give each machine a slice of the data.

SOLVES	COSTS	CONSISTENCY
Write throughput and storage beyond a single primary	Cross-shard queries and transactions, rebalancing, a far bigger operational surface	ACID within a shard; anything across shards needs extra design

⚠ THE PROBLEM

A multi-tenant SaaS primary is taking 60,000 writes per second on the largest instance the cloud offers, with 8 TB of data growing by 1 TB a month. Queries are already indexed and writes batched. Replicas won't help: each one would replay the same 60,000 writes per second.

How it works

Pick a **shard key** and route each row to the shard that owns that key. Each shard is an independent primary — usually with its own replicas — holding a fraction of the data and taking a fraction of the writes. Queries that include the shard key go to exactly one shard; queries that don't must *scatter* to every shard and *gather* the results.

The shard key is the decision you live with. It should have high cardinality, spread *load* (not just data) evenly, and match the dominant access pattern so most requests and transactions stay on one shard. For a SaaS product, `tenant_id` is often right: a tenant's data and transactions stay together. A timestamp is usually wrong — every new write hits the newest shard.

Plan for resharding on day one. A common technique maps keys to many **logical shards** (say 1,024) and logical shards to a handful of physical nodes; growing the cluster then moves whole logical shards rather than re-hashing every row. Live moves follow a copy, dual-write, verify, cut-over sequence — careful, slow work.

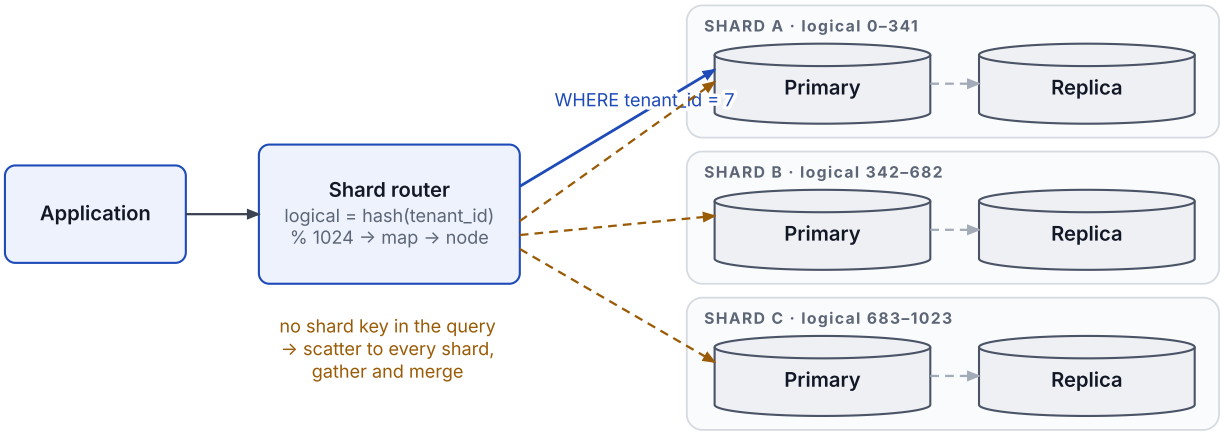


Figure 6.1 A router maps the shard key to a logical shard and then to a node. Queries with the key touch one shard; queries without it fan out to all of them.

ROUTING STRATEGIES

STRATEGY	STRENGTH	WEAKNESS
Hash-based	Even distribution, no lookup service	Range scans hit every shard; resizing moves data
Range-based	Efficient range scans, easy to split hot ranges	Sequential keys create a hot shard
Directory	Move any tenant anywhere, isolate a whale	The lookup service becomes a critical dependency

Follow-ups to expect

Q How do you run a report across all tenants?

Not on the OLTP shards. Stream changes (CDC) into a warehouse built for scans and aggregate there.

Q A transaction must span two shards. Now what?

First redesign so it doesn't — co-locate related data by the shard key. Otherwise use a saga; two-phase commit is rarely worth its blocking failure modes.

Q One tenant becomes 30% of all traffic.

Move it to a dedicated shard through the directory or logical-shard mapping, or sub-shard it by a secondary key. This is why indirection pays off.

Q How do you add a shard without downtime?

Move logical shards: copy a snapshot, stream changes, verify, pause writes for that slice briefly, flip the routing entry, then clean up.

REACH FOR IT WHEN

- Write throughput or storage exceeds one node — after indexing, batching and scaling up.
- Data has a natural owner (tenant, user, region).
- Blast-radius isolation per customer group is valuable.

WATCH OUT FOR

- Hot shards from celebrity tenants or sequential keys.
- Cross-shard joins and transactions — prefer sagas over 2PC.
- Global uniqueness and IDs: no auto-increment; use UUIDv7 or Snowflake-style IDs.

INTERVIEW CUE

"Read replicas don't fix a write bottleneck, because every replica replays every write. So we shard — and I'd justify the shard key by the most common query, not by what spreads rows evenly."

07 Consistent Hashing

Assign keys to nodes so that adding or removing one moves only a small slice of data.

SOLVES	COSTS	CONSISTENCY
Mass reshuffling of keys whenever cluster membership changes	Uneven load unless you use virtual nodes; membership must be agreed	None provided — it decides placement, not replication or agreement

⚠ THE PROBLEM

A cache tier spreads keys with `hash(key) % N`. Black Friday traffic arrives, the team adds a fifth node to the four already running — and about 80% of keys suddenly map to a different node. The hit rate collapses, every miss lands on the database, and the scaling action meant to relieve load causes an outage.

How it works

Hash both nodes and keys onto the same circular space — say 0 to 2^{32}. A key belongs to the first node found walking clockwise from the key's position. When a node joins, it takes over only the keys between itself and its predecessor; when one leaves, only its keys move, to its neighbour. On average roughly 1/N of keys move, instead of almost all of them.

With a few nodes, the arcs between them are uneven, so some nodes own far more keys than others. **Virtual nodes** fix this: each physical server is placed at 100–200 points on the ring. Load evens out statistically, a failed node's keys scatter across many survivors rather than doubling one neighbour's load, and bigger machines can simply receive more points.

Systems like Dynamo and Cassandra add replication on top — store each key on the next R distinct physical nodes clockwise. Alternatives worth naming: **rendezvous hashing** (highest random weight) gives the same minimal movement without a ring; **jump consistent hash** is tiny and fast when nodes are numbered and only appended.

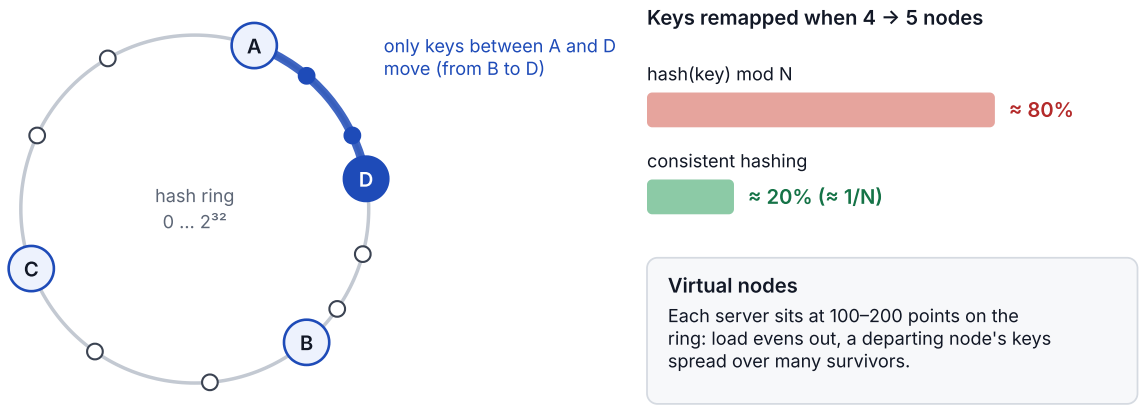


Figure 7.1 Adding node D on the ring only reassigns keys on the arc between A and D. Modulo hashing, by contrast, remaps most keys when N changes.

Follow-ups to expect

Q How many virtual nodes?

Typically 100–256 per server — enough to keep load within a few percent of the mean without making the ring lookup large.

Q How do clients agree on the ring?

Membership comes from a coordination service (etcd, ZooKeeper) or gossip; clients cache and refresh it. In a cache, disagreement costs misses, not corruption.

Q What about one extremely hot key?

Hashing can't split a single key. Replicate it under suffixed names and read one at random, or put an in-process cache in front.

Q Ring or rendezvous hashing?

Both move about 1/N of keys. Rendezvous needs no ring or vnodes but costs O(N) per lookup — fine for tens of nodes.

REACH FOR IT WHEN

- Client-side sharding of a distributed cache.
- Partitioned key-value stores with elastic membership.
- Sticky routing of users or sessions to backends.

WATCH OUT FOR

- Hot keys — hashing spreads keys, not popularity.
- Too few virtual nodes and uneven arcs.
- Clients with different views of membership routing differently.

INTERVIEW CUE

"It's a placement technique: it minimizes data movement when nodes change. It doesn't give me replication, failure detection or consistency — those I still have to design."

PART III OF VI

III

Asynchronous work & decoupling

A synchronous request is a promise: everything it touches must be up, fast and correct right now. Moving work off the request path breaks that chain — the user gets an answer sooner, a slow email provider no longer fails a checkout, and spikes are absorbed instead of passed along. The price is that "done" now happens later, possibly twice.

THE QUESTION THIS PART ANSWERS

What must be true before we answer the user — and how do we make sure everything else eventually happens, exactly as if once?

08 **Async processing**
Split the critical path from the rest.

09 **Message queues**
Buffer work and hand each item to one consumer.

10 **Pub/Sub**
Deliver one event to many independent consumers.

11 **Event-driven architecture**
Couple services through facts, not calls.

12 **Job queues**
Durable, retryable, schedulable units of work.

13 **Background workers**
The fleet that executes those jobs.

14 **Workflow engines**
Long-running, multi-step processes with state.

→ **Common thread**
At-least-once delivery means idempotent consumers.

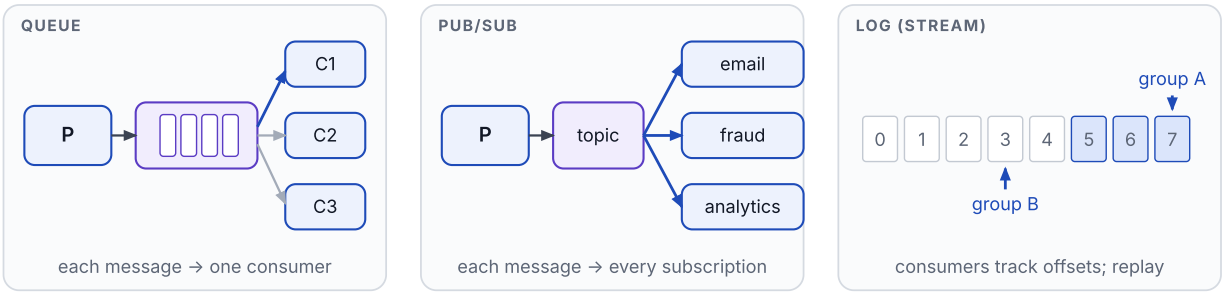


Figure III Three delivery shapes. A queue distributes work; pub/sub copies each message to every subscription; a log keeps messages and lets each consumer group keep its own position.

08 Async Processing

Answer the user once the essential work is durable; do everything else afterwards.

SOLVES	COSTS	CONSISTENCY
Slow, fragile responses caused by work the user isn't waiting for	Eventual consistency, new infrastructure, harder tracing	Side effects complete seconds or minutes after the response

⚠ THE PROBLEM

The checkout endpoint validates the cart, saves the order, authorizes payment, sends a confirmation email, renders an invoice PDF and records analytics — all before responding. p99 is 2.5 seconds, and when the email provider had an outage last month, checkout failed with it. Customers were unable to pay because a notification couldn't be sent.

How it works

Draw a line through the request. Above it is the **critical path**: whatever must be true before you can honestly say "your order is placed" — validation, persisting the order, authorizing payment. Below it is everything else. Persist a record of the deferred work durably, respond, and let workers do the rest. Response time drops to the critical path, and the deferred dependencies stop being able to fail the request.

The trap is the **dual write**. If the code commits the order and then publishes a message, a crash between the two loses the message forever; publish first and a rollback leaves a phantom event. The fix is the **transactional outbox**: write the event into an `outbox` table in the *same* database transaction as the order, then have a relay (polling, or change-data-capture) publish it and mark it sent. It may publish twice; it will never publish zero times.

Fire-and-forget inside the web process — an unawaited promise, a thread pool — is not async processing. It disappears on every deploy and crash. And give users an honest status: "Invoice is being prepared" with a link that updates beats a spinner that silently gave up.

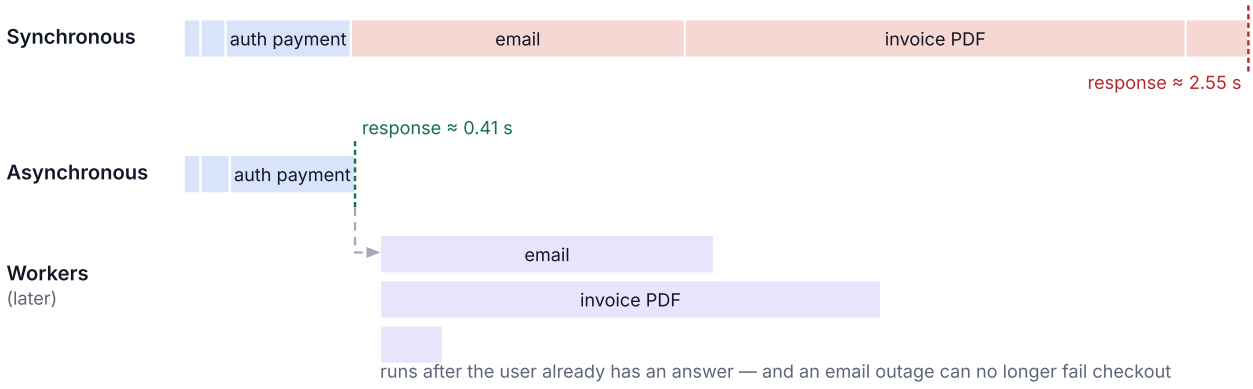


Figure 8.1 Same work, different shape. The user waits only for the critical path; the deferred work runs on workers, independently and in parallel.

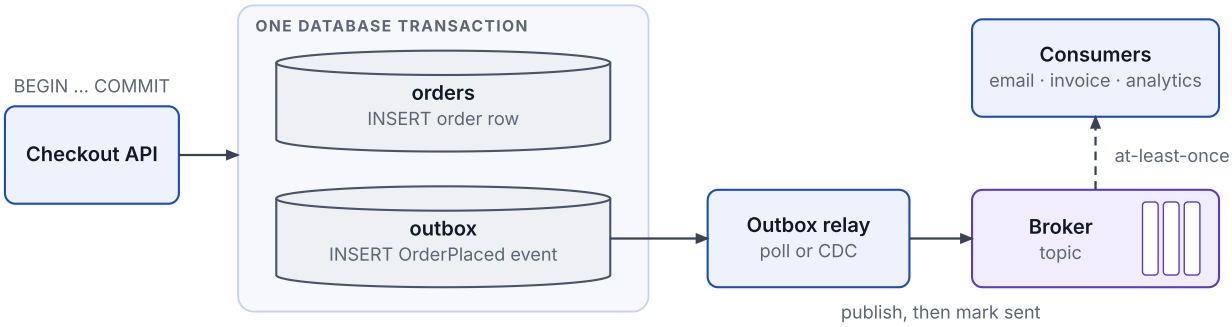


Figure 8.2 The transactional outbox. The order and its event commit atomically; a relay publishes afterwards, so a crash can delay the event but never lose it.

REACH FOR IT WHEN

- Work isn't needed to form the response.
- A slow or unreliable third party sits on the request path.
- The work is bursty and can be smoothed out.

WATCH OUT FOR

- Dual writes without an outbox.
- Users who need the result now — don't defer it just because you can.
- Lost traceability; carry a correlation ID into every message.

INTERVIEW CUE

"I'd split the request into what must happen before we respond and what can happen later — and publish the 'later' part through an outbox so a crash can't lose it."

09 Message Queues

A durable buffer that lets producers and consumers work at their own pace.

SOLVES	COSTS	CONSISTENCY
Traffic spikes, mismatched processing rates, direct coupling	Duplicates, limited ordering, another system to operate	At-least-once delivery is the practical default

⚠ THE PROBLEM

Photo uploads spike 20× when a marketing email lands. Thumbnail generation runs inside the upload request, so API servers saturate, requests time out, and users retry — making it worse. The thumbnails don't need to exist within 200 ms; they need to exist within a minute.

How it works

The producer sends a message to a queue and moves on. Consumers *receive* messages; a received message is not deleted but becomes invisible for a **visibility timeout** (a lease). If the consumer finishes, it acknowledges and the message is deleted. If it crashes or the lease expires, the message reappears for another consumer. After too many attempts, it moves to a **dead-letter queue** for inspection instead of blocking everyone forever.

Multiple consumers on one queue compete: each message goes to one of them. That makes a queue a natural **load leveller** — the backlog grows during the spike and drains afterwards at the rate your consumers can sustain, while producers stay fast.

Delivery semantics come in three flavours. **At-most-once** acks before processing and can lose messages. **At-least-once** acks after processing and can duplicate them — this is what nearly everyone runs. **Exactly-once** is, in practice, at-least-once delivery plus idempotent processing. Ordering is similar: standard queues are best-effort; FIFO queues order per message group, which caps parallelism at the number of groups.

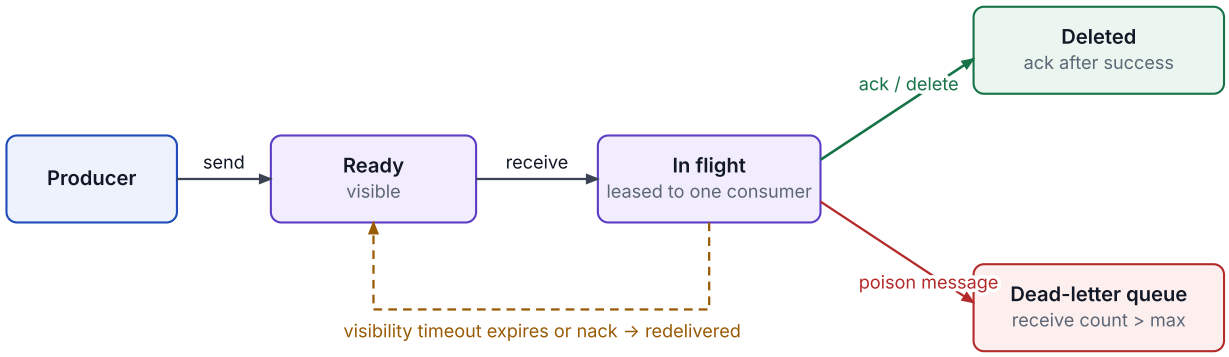


Figure 9.1 A message's lifecycle. The visibility timeout is what turns a consumer crash into a redelivery rather than a loss — and what produces duplicates.

Follow-ups to expect

Q How do you keep per-customer order and still scale?

Partition by key — FIFO message groups or Kafka partitions keyed by customer. Order holds within a key; parallelism comes from many keys.

Q How do you size the visibility timeout?

Several times the p99 processing time, plus a heartbeat that extends it for long jobs. Too short creates duplicates; too long delays crash recovery.

Q The backlog grows and never drains. What do you check?

Arrival versus processing rate, consumer errors, a poison message cycling, and whether a downstream limit caps consumers — then scale or shed.

Q SQS, RabbitMQ or Kafka?

SQS for managed, simple work queues; RabbitMQ for flexible routing and low latency; Kafka for retention, replay and very high throughput with per-partition order.

REACH FOR IT WHEN

- Producers burst faster than consumers can process.
- Each unit of work needs exactly one handler.
- You want workers to scale on backlog.

WATCH OUT FOR

- Visibility timeout shorter than processing time — concurrent duplicates. Extend leases on long jobs.
- Large payloads: store the blob, send a reference (claim check).
- A growing backlog is a silent outage; alert on *oldest message age*.

INTERVIEW CUE

"At-least-once means my consumer must be idempotent. And a queue hands each message to one worker — if several systems need the same message, that's pub/sub."

10 Pub/Sub

Publish once; let every interested system receive its own copy.

SOLVES	COSTS	CONSISTENCY
One event that many independent systems must react to	Lag monitoring per subscriber, schema discipline, retention planning	Each subscriber converges on its own schedule

⚠ THE PROBLEM

When an order is placed, fulfillment, email, analytics, fraud and loyalty all need to know. The order service calls each of them in turn. Any one being slow slows checkout, and adding a sixth consumer means changing and redeploying the order service — which has become the place everyone's integration lives.

How it works

The producer publishes to a **topic**. Each **subscription** receives its own copy of every message and keeps its own position, so a slow analytics pipeline never holds up fulfillment. New consumers attach themselves; the producer doesn't know or care how many there are.

Two families exist. **Push-style** brokers (SNS fanning out to SQS queues, Google Pub/Sub) copy each message into per-subscription queues and delete it once acknowledged. **Log-based** systems (Kafka, Kinesis, Pulsar) append messages to a retained, partitioned log; each consumer group tracks an offset and can rewind to **replay** history — invaluable when a new consumer needs the past, or a bug requires reprocessing.

Within one subscription, consumers usually still compete for messages (a consumer group), so pub/sub and queuing combine naturally: fan out across systems, load-balance within each. In log systems, ordering is guaranteed per partition, so key messages by entity (`order_id`) to keep each order's events in sequence.

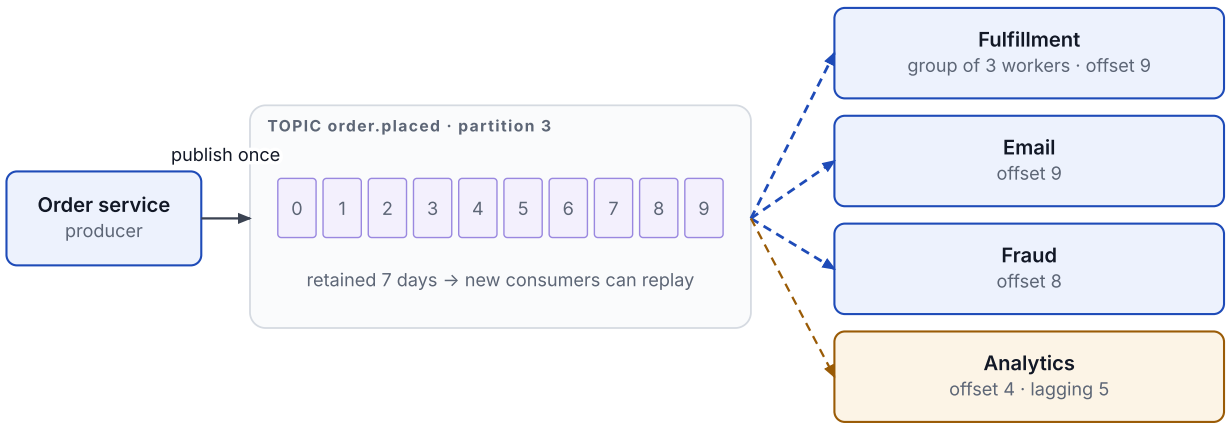


Figure 10.1 One publish, four independent consumers. Each subscription advances at its own pace; analytics lagging five messages affects no one else.

✓ REACH FOR IT WHEN

- Several systems react to the same business event.
- Consumers should be added without touching the producer.
- Replay or audit of past events is valuable (log-based).

! WATCH OUT FOR

- Retention shorter than your worst consumer outage.
- Breaking schema changes — use a registry and evolve compatibly.
- A poison message stalling a whole log partition.

INTERVIEW CUE

"One task done by one worker suggests a queue. One fact that many independent systems react to suggests pub/sub."

11 Event-Driven Architecture

Services announce facts about what happened; others decide how to react.

SOLVES	COSTS	CONSISTENCY
Chains of synchronous calls that couple services' uptime and releases	Flows that are hard to see end to end; eventual consistency in the UI	Eventual across services; strong only inside each service

⚠ THE PROBLEM

Order calls Inventory, which calls Payment, which calls Shipping, which calls Notifications. Five services at 99.9% availability give the chain roughly 99.5%. Latencies add up, a release of any one can break the flow, and every new requirement edits the Order service.

How it works

Each service publishes **events** — immutable facts in the past tense, like `OrderPlaced` or `PaymentCaptured` — that it owns. Other services subscribe and react. Contrast this with a **command** such as `ChargeCard`, which is addressed to one handler and can be refused. Events say what happened; commands ask for something to happen.

When services react to each other's events with no central coordinator, that is **choreography**: loosely coupled and easy to extend, but beyond four or five steps the overall flow becomes hard to see.

Orchestration puts the flow in one place (see Pattern 14). Decide what goes in the payload too: *thin* events carry IDs and force consumers to call back; *fat* events carry the state consumers need and make them self-sufficient, at the cost of a larger contract.

Treat events as a public API. Give every event an ID for de-duplication, a version, a timestamp, and correlation and causation IDs so one trace can follow a business flow across services. Event sourcing — storing events as the source of truth — is a separate, much bigger decision.

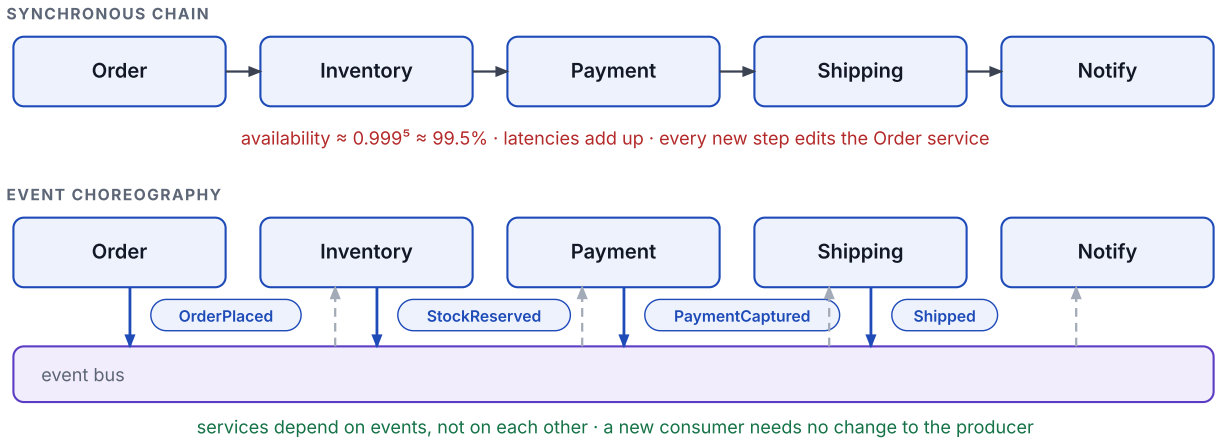


Figure 11.1 The same flow as a call chain and as choreography. Services depend on the event contracts, not on each other's availability.

```
// A well-formed event envelope
{ "event_id": "01J9Z6K1QH…",  "type": "order.placed",  "version": 3,
  "occurred_at": "2026-10-13T09:41:07Z",  "producer": "order-service",
  "correlation_id": "chk-5531",  "causation_id": "req-88f2",
  "data": { "order_id": 5531, "customer_id": 912, "total_cents": 4999, "currency": "USD" } }
```

Follow-ups to expect

Q How do you change an event schema safely?

Additive, backward-compatible changes only; consumers ignore unknown fields. For breaking changes, publish a new version alongside and migrate consumers first.

Q How do you debug a flow across six services?

Propagate correlation IDs into every event, trace across producers and consumers, and keep an event catalog naming owners and subscribers.

Q Choreography or orchestration?

Choreography for a few loosely related reactions; orchestration once a process has ordering, deadlines and compensation someone must be able to see.

Q Isn't the UI now inconsistent?

Briefly, yes — so design for it: explicit pending states, optimistic UI, and reads from the owning service rather than a lagging projection.

REACH FOR IT WHEN

- Multiple domains react to the same business facts.
- Teams need to deploy independently.
- An audit trail of what happened has value.

WATCH OUT FOR

- "Who listens to this?" — keep an event catalog with owners.
- Events used as disguised RPC calls, expecting a reply.
- UI that assumes downstream effects are instant.

INTERVIEW CUE

"I treat an event as a durable business fact with an owner and a versioned schema — not as another HTTP call that happens to go through a broker."

12 Job Queues

Represent deferred work as durable jobs — with retries, schedules and priorities built in.

SOLVES	COSTS	CONSISTENCY
Reliable deferred, delayed and scheduled execution	Job state to manage; duplicates on every crash	Job outcomes are eventual; handlers must be idempotent

⚠ THE PROBLEM

The product needs password-reset emails within seconds, a reminder 24 hours after signup, nightly invoice generation, and bulk newsletters that must never delay the password resets. Today this is a mix of cron scripts and in-process timers that vanish on deploy.

How it works

A job is a named operation with the *minimum* data needed to run it — IDs, not full objects, because data changes and the handler should read fresh state. Around it the queue keeps metadata: attempts, `run_at`, priority, deadline. Libraries such as BullMQ, Sidekiq and Celery sit on Redis or a broker; Postgres-backed queues (graphile-worker, pg-boss) let you enqueue *in the same transaction* as your business write, which solves the dual-write problem for free.

A job moves through states: delayed, waiting, active, then completed — or failed and rescheduled with backoff until its attempts run out, when it is dead-lettered. Use separate queues with weighted workers rather than strict priorities, which can starve low-priority work indefinitely.

The question to always ask: **what happens if the worker crashes after the side effect but before acknowledging?** The job runs again. Make handlers idempotent: pass an idempotency key to external APIs, guard internal effects with unique constraints, and record completion in the same transaction as the effect.

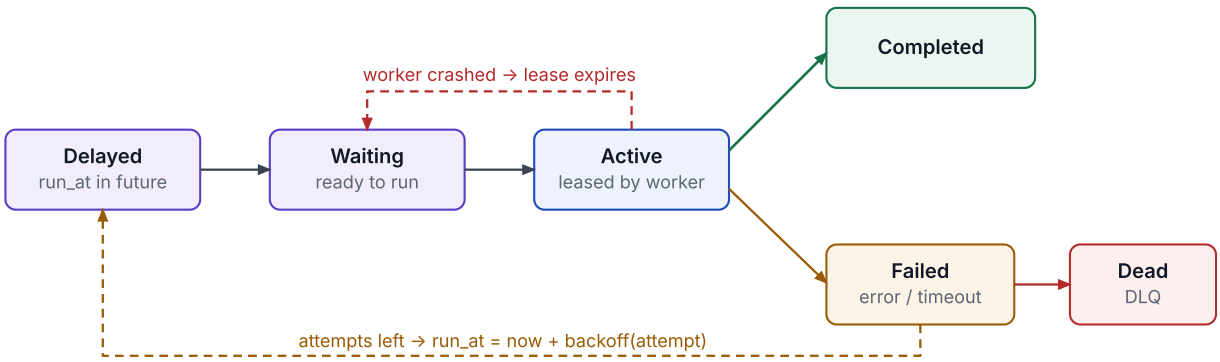


Figure 12.1 Job lifecycle. Two paths lead back to the start: a failure with attempts left, and a crash that lets the lease expire.

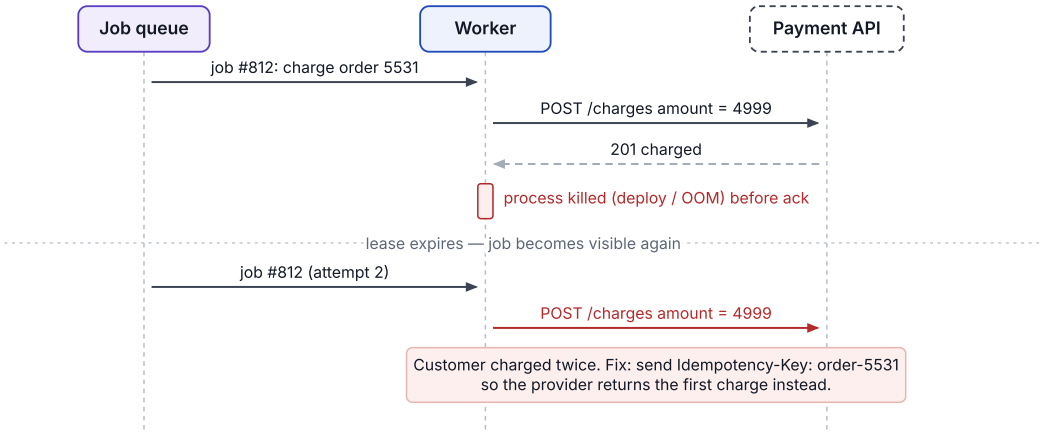


Figure 12.2 The crash window. The provider charged the card, but the queue never heard about it — so the job runs again.

REACH FOR IT WHEN

- Work must happen reliably later, on a schedule, or after a delay.
- You need retries, priorities and visibility into failures.
- Calls to rate-limited APIs must be throttled.

WATCH OUT FOR

- Poison jobs retried forever — cap attempts.
- Jobs referencing data that was deleted meanwhile.
- Track depth, processing rate and *oldest job age*.

INTERVIEW CUE

"What happens if the worker dies after charging the card but before acking the job? That's the case my idempotency design has to cover."

13 Background Workers

Dedicated processes that do deferred work, sized and scaled apart from the API.

SOLVES	COSTS	CONSISTENCY
Heavy or slow work degrading request latency for everyone	Another fleet to deploy, scale, observe and shut down cleanly	Inherited from the queue: expect redelivery

⚠ THE PROBLEM

Invoice PDFs are rendered inside the Node.js API. Each render holds the event loop for 300 ms, so every unrelated request on that instance stalls behind it. At month end, API p99 triples — not because of API traffic, but because of work the API should never have been doing.

How it works

Workers **pull** jobs from the queue, process them with bounded concurrency, and acknowledge. Pulling gives natural backpressure: a worker only takes what it can handle. Scale the pool on queue depth or, better, the age of the oldest job — that is what users actually feel — but cap it by what downstream systems can absorb. Doubling workers can simply double the load on a database that is already the bottleneck.

Separate pools by workload class. CPU-heavy rendering and I/O-heavy webhook delivery have different ideal concurrency, and sharing a pool lets a flood of one starve the other. Give every job a timeout, extend leases for long-running jobs with heartbeats, and recycle processes that leak memory.

Shutdown is where most worker bugs live. On `SIGTERM` a worker should stop fetching, finish or release in-flight jobs within the grace period, and exit before the orchestrator sends `SIGKILL`. Anything still running at that moment will be redelivered — which is fine, as long as the handlers are idempotent.

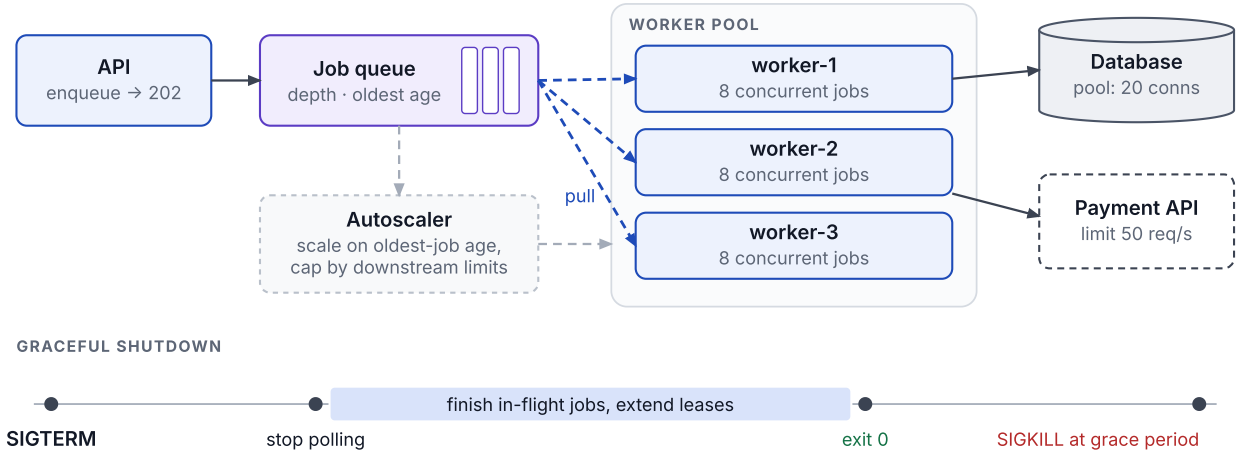


Figure 13.1 A worker pool pulls from the queue, scales on job age, and respects downstream limits. Below, the graceful-shutdown sequence every deploy depends on.

Follow-ups to expect

Q How do you stop one job type starving another?

Separate queues and pools per class, or weighted fetching. Never one FIFO shared by two-second and two-hour jobs.

Q How many workers do you need?

Little's law: workers × concurrency ≥ arrival rate × processing time, plus headroom — then cap by what the database and third parties tolerate.

Q A job runs for an hour. How does the queue know it's alive?

The worker heartbeats to extend its lease and checkpoints progress, so a restart resumes instead of starting over.

Q Containers or serverless functions?

Functions suit short, spiky jobs. Long, CPU-heavy or connection-hungry jobs fit long-lived containers with stable database pools.

REACH FOR IT WHEN

- Work is CPU-heavy, long-running or I/O fan-out.
- Throughput must scale independently of web traffic.
- Work needs isolation from request latency.

WATCH OUT FOR

- Stuck jobs with no timeout holding concurrency slots.
- Duplicate execution on every deploy.
- Autoscaling that overwhelms the database or a rate-limited API.

INTERVIEW CUE

"The queue stores work; the worker executes it. I'd scale workers on the age of the oldest job, capped by what the downstream can take."

14 Workflow Engines

When a process spans many steps and days, persist where it is — not just what's next.

SOLVES	COSTS	CONSISTENCY
Long-running, multi-step processes that must survive crashes	A new platform, determinism rules, versioning discipline	Steps commit independently; compensations restore invariants

⚠ THE PROBLEM

Loan onboarding runs an identity check, waits days for the customer to upload documents, runs a credit check, waits for a human approval, then opens the account. Today it is a `status` column, five cron jobs and a chain of queues. Records get stuck in states nobody expected, and no one can answer "where is application 4471, and why?"

How it works

You describe the process as a workflow — in code with Temporal or similar, or as a state machine with AWS Step Functions or Camunda. The engine durably records each step's result. Activities (the real side effects) run on workers with their own retries and timeouts; durable **timers** can wait three days without holding anything in memory; **signals** deliver human decisions. After a crash, the engine resumes from the recorded history exactly where it stopped.

The core distributed pattern here is the **saga**: a sequence of local transactions, each paired with a compensating action. If step three fails, the engine runs the compensations for steps two and one, in reverse. Compensation is semantic undo — a refund, not a rollback — so design each one as a real business operation.

Code-first engines replay history to rebuild state, so workflow code must be deterministic: no clocks, random numbers or network calls outside activities. Versioning matters too — a workflow started last month must finish under the logic it started with.

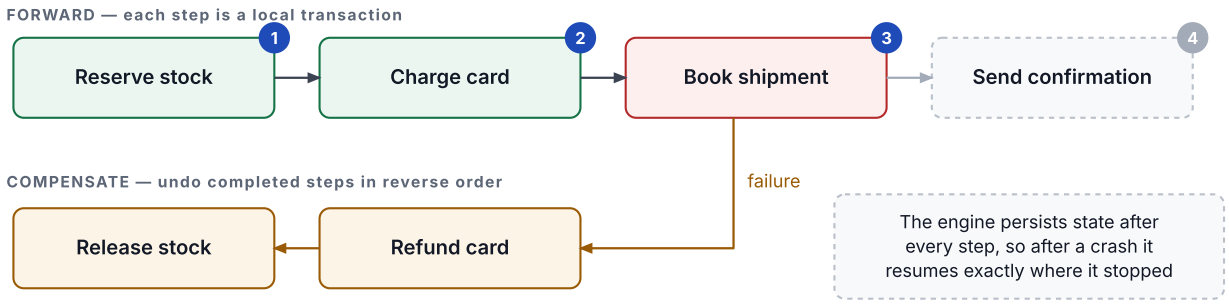


Figure 14.1 A saga. Shipment booking fails, so the engine refunds the card and releases stock, in reverse order of the completed steps.

	JOB QUEUE	WORKFLOW ENGINE
Unit of work	One independent task	A whole process of many steps
State	Lives in your tables and ad-hoc flags	Persisted by the engine, step by step
Waiting days	Cron jobs and polling	Durable timers and signals
Failure midway	You hand-write cleanup	Retries plus modelled compensations

Follow-ups to expect

Q Isn't this just a state column in a table?

Until you need timers, per-step retries, visibility, versioning and crash recovery — then you are writing a workflow engine yourself, usually badly.

Q What if a compensation fails?

Retry it — compensations must be idempotent. If it still fails, escalate to a human queue; never drop it silently.

Q How do you deploy while thousands of workflows are in flight?

Version the workflow: running instances finish on the old path, new ones start on the new. Engines provide version markers for exactly this.

Q Temporal or Step Functions?

Temporal: workflows as ordinary code, portable and self-hostable. Step Functions: managed JSON state machines, tightly integrated with AWS.

REACH FOR IT WHEN

- Several durable steps, long waits or human actions.
- You must know exactly where each instance stands.
- Partial failure needs compensation, not just retry.

WATCH OUT FOR

- Using it for one-step jobs a queue handles fine.
- Non-deterministic workflow code; unversioned changes.
- The engine becoming a critical shared dependency.

INTERVIEW CUE

"If I need to know exactly where a multi-step process stopped, and resume it after a crash, I'd use a workflow engine instead of chaining queues together."

PART IV OF VI

IV

Designing for failure

In a distributed system, partial failure is the normal state. Something is always slow, restarting or half-broken. The four patterns in this part are not optional extras; they are the minimum safety equipment for any call that crosses a network — and they only work when used together.

THE QUESTION THIS PART ANSWERS

For every network call: what if it times out, what if it runs twice, and what if the dependency is down for ten minutes?

15 **Timeouts**
Bound how long any call can wait.

16 **Retries with backoff**
Recover from transient failures safely.

17 **Circuit breakers**
Stop calling something that is clearly broken.

18 **Idempotency**
Make repeated attempts harmless.



Figure IV How the four compose around one call. The deadline bounds everything; the breaker decides whether to try; retries repeat with jittered backoff; each attempt has its own timeout; and the idempotency key, identical on every attempt, makes repetition safe.

PATTERN	ANSWERS THE QUESTION	WITHOUT IT
Timeout	How long will I wait?	Hung calls exhaust threads, sockets and pool slots
Retry + backoff	Should I try again, and when?	Transient blips become user-facing errors — or retry storms
Circuit breaker	Should I try at all?	A dead dependency drags every caller down with it
Idempotency	Is it safe to try again?	Retries double-charge, double-ship, double-send

15 Timeouts

Never wait forever — and remember the other side may still be working.

SOLVES	COSTS	CONSISTENCY
Resource exhaustion and cascading failure from hung dependencies	False failures if set too tight; tuning per call	The outcome of a timed-out call is unknown, not failed

⚠ THE PROBLEM

The recommendations service doesn't fail — it just starts answering in 30 seconds. The product page calls it with the HTTP client's default, which is no timeout. Within minutes every connection and request slot on the product service is waiting on recommendations, and the whole storefront is down because of a widget most users never scroll to.

How it works

Set an explicit bound on every network call. Distinguish a short **connect timeout** (hundreds of milliseconds — a healthy host accepts fast) from the **request timeout**, and choose the latter from data: a little above the dependency's p99 or p99.9 for that call, and always within the caller's own budget. In Node.js, `AbortSignal.timeout(ms)` on `fetch` is the simplest correct tool.

Better still, propagate a **deadline**. The edge sets "this request must finish by T", and each hop passes the remaining budget downstream — gRPC does this natively; over HTTP, use a header. A service with 20 ms left should not start a 200 ms query; the caller has already given up, and that work is pure waste during the exact moment the system is overloaded.

A timeout tells you that *you* stopped waiting. It does not tell you the operation didn't happen. The server may have committed a millisecond after you gave up. That's why a timeout and a blind retry together can duplicate side effects — the problem idempotency (Pattern 18) exists to solve. Pair a timeout with a fallback where one exists: hide the widget, serve a cached default.

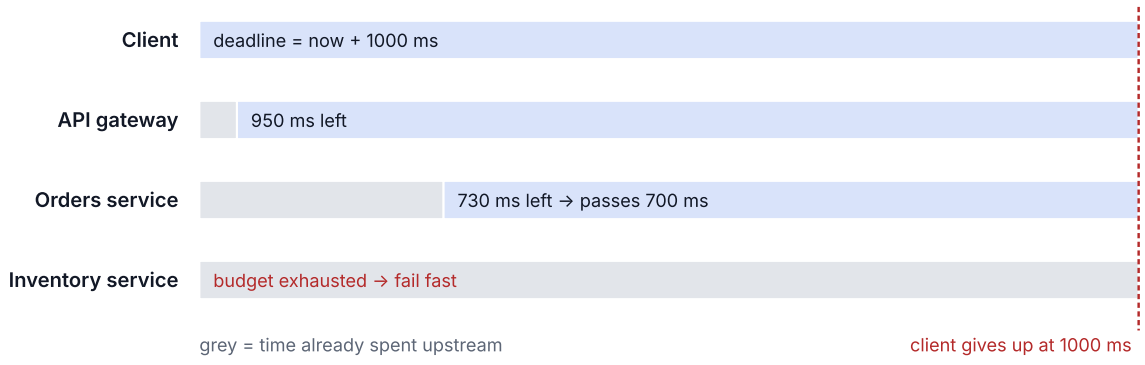


Figure 15.1 Deadline propagation. Each hop passes on what is left of the budget, and a service that receives an exhausted budget fails fast instead of doing work nobody will read.

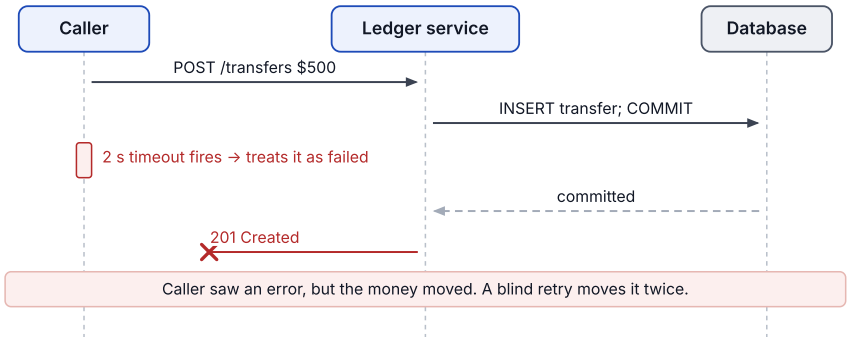


Figure 15.2 Timed out is not the same as failed. The caller saw an error; the ledger committed the transfer.

REACH FOR IT WHEN

- Always — every network call, database query and lock wait.
- Especially on non-critical dependencies that can be skipped.

WATCH OUT FOR

- Library defaults that are infinite or minutes long.
- Retries × timeout exceeding the caller's own timeout.
- A downstream timeout longer than the upstream one.

INTERVIEW CUE

"A timeout bounds how long I wait; it doesn't tell me whether the operation happened. So anything I'd retry after a timeout has to be idempotent."

16 Retries with Backoff

Retry transient failures — slowly, randomly, and a bounded number of times.

SOLVES	COSTS	CONSISTENCY
Transient faults: blips, throttling, failovers, brief overload	Extra load at the worst moment; duplicate side effects	Safe only for idempotent operations

⚠ THE PROBLEM

A database failover takes eight seconds. Two thousand application instances retry immediately in a tight loop. The new primary comes up and is instantly hit by a synchronized wall of reconnects and queries — and falls over again. The retries meant to ride out the incident are now prolonging it.

How it works

Retry only what is likely to succeed next time: connection resets, timeouts on idempotent calls, `429`, `502`, `503`, `504`. Never retry `400`, `401`, `403`, `404` or validation errors — they will fail identically, just more expensively. Honour `Retry-After` when a server sends it.

Grow the delay exponentially — `min(cap, base × 2ⁿ)`: 100, 200, 400, 800 ms, capped — so a struggling dependency gets room to recover. Then add **jitter**. Without it, every client that failed together retries together, recreating the spike on a schedule. With *full jitter*, each client sleeps a random time between zero and the backoff value, and the load spreads into a smooth trickle.

Bound retries by both attempts and the overall deadline, and retry at **one layer only**. Three layers each retrying three times turn one user click into 27 database queries during an outage. For large fleets, a **retry budget** — retries may add at most, say, 10% to normal traffic — stops retries from ever becoming the dominant load.

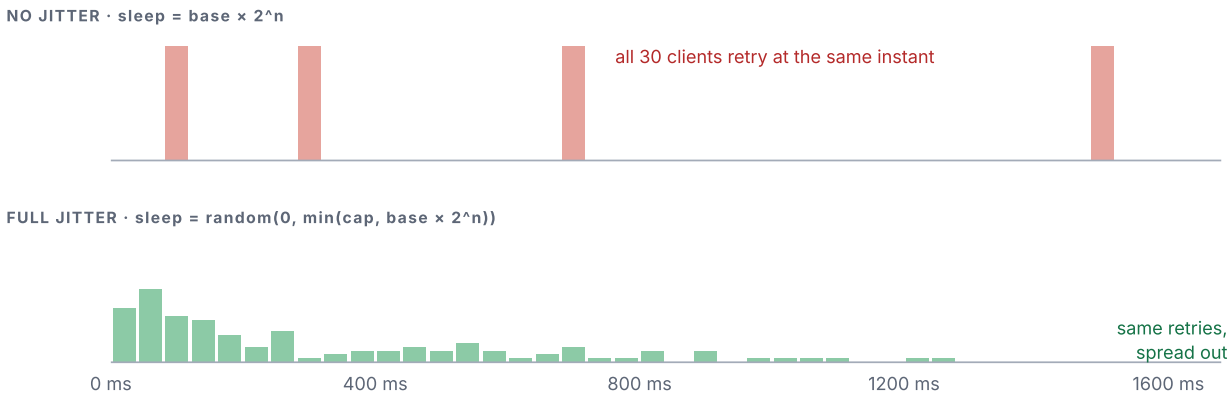


Figure 16.1 Thirty clients, four attempts each. Without jitter, retries land as synchronized spikes; with full jitter, the same retries arrive spread out.

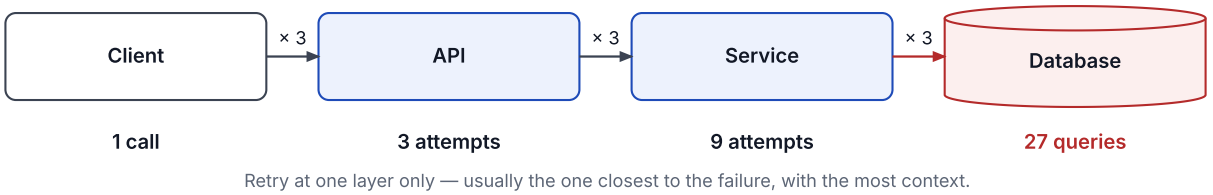


Figure 16.2 Retry amplification. Nested retries multiply: three layers of three attempts become 27 queries against the component that is already failing.

Follow-ups to expect

Q Where should retries live?

At one layer — usually the client closest to the failure, which knows whether it is transient — and nowhere else in the chain.

Q Can you retry a POST?

Only with an idempotency key. Otherwise a timeout followed by a retry can duplicate the effect.

Q What limits make sense?

Interactive paths: two or three attempts, total delay well inside the deadline. Queued work can retry for minutes or hours with a larger cap.

Q How do retries interact with a circuit breaker?

The breaker wraps the retry loop: when open, nothing is retried; failed attempts count toward opening it.

REACH FOR IT WHEN

- Failures are plausibly transient.
- The operation is idempotent or carries an idempotency key.
- There is time left in the caller's deadline.

WATCH OUT FOR

- Retry storms from immediate, unjittered retries.
- Retrying permanent errors or non-idempotent writes.
- No defined behaviour after the last attempt — fallback, DLQ or clear error.

INTERVIEW CUE

"I'd state what we retry, how often, how many times, and what happens after the last attempt — exponential backoff with full jitter, at one layer, inside the request deadline."

17 Circuit Breakers

Stop calling a dependency that is clearly failing, and check back periodically.

SOLVES	COSTS	CONSISTENCY
Wasted resources and extra load aimed at a failing dependency	Threshold tuning; may reject calls that would have succeeded	Not applicable — but fallbacks may serve stale data

⚠ THE PROBLEM

The fraud-scoring service is down. Each checkout waits for a two-second timeout, retries three times, and only then proceeds without a score. Checkout threads are now tied up for six seconds each, throughput collapses, and the fraud service — trying to restart — is flooded by every caller the moment it comes back.

How it works

Wrap calls to each dependency in a small state machine. **Closed** is normal: calls pass through and outcomes are recorded in a sliding window. When failures cross a threshold — for example, at least 50% of at least 20 calls in ten seconds — the breaker **opens**, and calls fail immediately without touching the network. After a cool-down it goes **half-open** and lets a few trial calls through: if they succeed it closes; if any fails it opens again.

Failing fast does two things. It gives callers their threads, sockets and latency budget back at once, and it gives the sick dependency room to recover instead of a thundering herd. What happens while open is a product decision: serve a cached value, a sensible default, queue the work for later, hide the feature, or return a clear error.

Keep one breaker per dependency — often per host or endpoint, so one bad instance doesn't block healthy ones. Pair breakers with **bulkheads** (separate connection pools per dependency) so no single dependency can consume them all. Libraries: opossum for Node.js, resilience4j for Java, Polly for .NET; meshes like Envoy offer similar outlier detection.

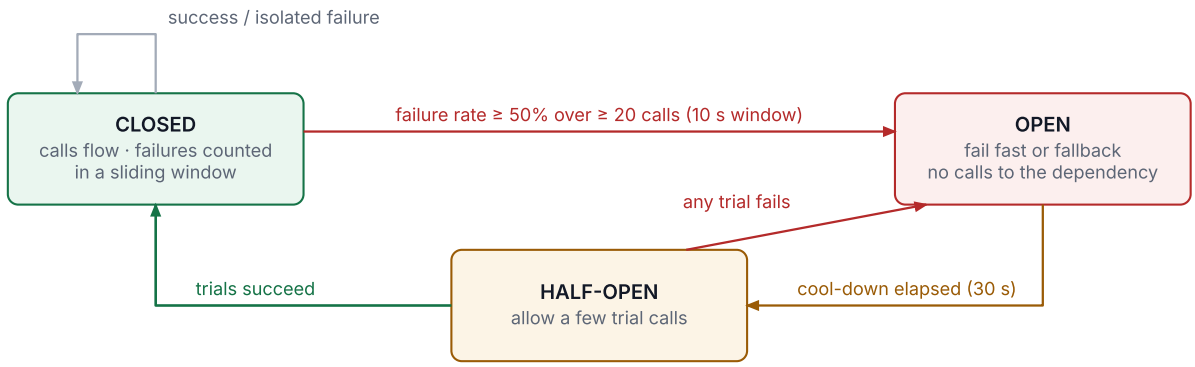


Figure 17.1 Breaker states. The minimum-volume rule matters — two failures out of three calls at 3 a.m. shouldn't open the circuit.

✓ REACH FOR IT WHEN

- A remote dependency can stay unhealthy for minutes.
- A useful fallback or degraded mode exists.
- Callers share scarce resources such as thread or connection pools.

! WATCH OUT FOR

- Over-sensitive thresholds that flap open and closed.
- One shared breaker across healthy and unhealthy hosts.
- Open breakers hiding outages — alert on state changes.

INTERVIEW CUE

"A timeout limits how long a call waits; a circuit breaker decides whether to make the call at all."

18 Idempotency

Make "do it again" safe, so retries and duplicates can't double-charge anyone.

SOLVES	COSTS	CONSISTENCY
Duplicate side effects from retries and redelivered messages	Storing keys and results; careful atomicity	Turns at-least-once delivery into exactly-once effects

⚠ THE PROBLEM

A mobile app sends `POST /payments`. The server charges the card and commits, but the response is lost on a flaky train connection. The app times out and retries, as it should. Without protection, the customer is charged twice — and the support ticket arrives before the engineering team knows anything went wrong.

How it works

The client generates an **idempotency key** — a UUID — for each logical operation and sends the same key on every retry of it. The server atomically inserts the key, scoped to the caller, with status `in_progress` and a fingerprint of the request body. If the insert succeeds, it does the work, stores the response, and marks the key complete. If the key already exists and is complete, it returns the stored response without doing anything again.

The insert must rely on a unique constraint, not "check, then insert" — two concurrent retries would both pass the check. Return a conflict for a key that is still in progress, and an error for the same key with a different body. Keep keys longer than any client will plausibly retry; 24 hours is common for APIs.

Many operations can be made **naturally idempotent**: `PUT` that sets state, upserts on a business key, conditional updates on a version number. For queue consumers, record processed message IDs in the same transaction as the effect. When the side effect is external, pass a key downstream too — payment providers accept one for exactly this reason.

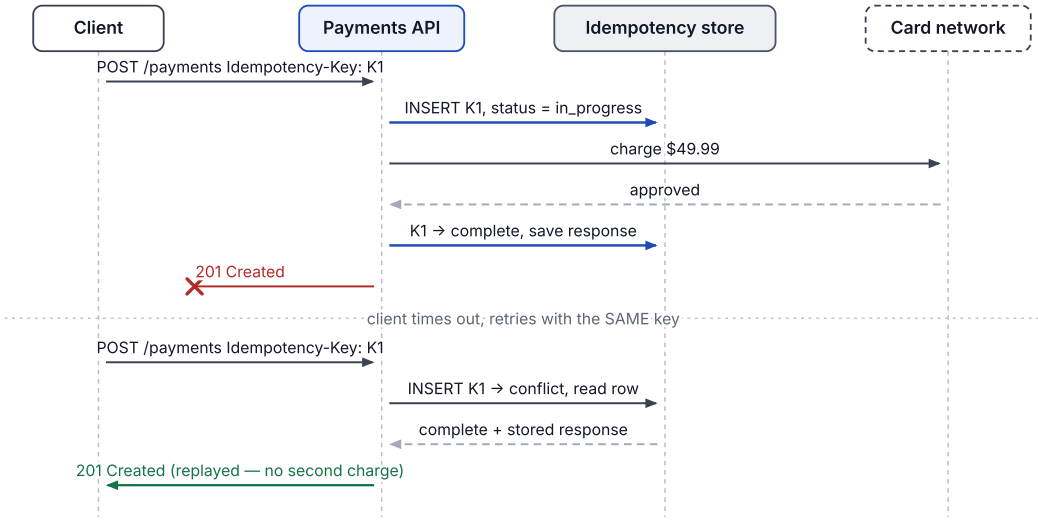


Figure 18.1 The lost-response retry. The second request finds key K1 complete and replays the original response instead of charging again.

SERVER BEHAVIOUR BY KEY STATE

INCOMING KEY IS...	SERVER RESPONDS WITH
New	Insert as `in_progress`, execute, store response, mark complete
Complete, same body	The stored response, unchanged — nothing executes
In progress	`409 Conflict` — the client retries later with the same key
Complete, different body	`422` — key reused for a different operation, a client bug

REACH FOR IT WHEN

- Payments, orders, transfers — any POST that may be retried.
- Every queue or event consumer.
- Any operation with an external side effect.

WATCH OUT FOR

- Keys generated per attempt rather than per operation.
- Key store separate from the effect, so they can disagree.
- Retention shorter than the client's retry window.

INTERVIEW CUE

"What happens if the server succeeds but the client times out before receiving the response? An idempotency key, stored atomically with the result, is the answer."

PART V OF VI

V

Real-time delivery

HTTP was designed for clients that ask. Real-time features need servers that tell. The three techniques here trade simplicity, infrastructure compatibility and directionality against each other — and the right answer is usually the simplest one that meets the latency the product actually needs.

THE QUESTION THIS PART ANSWERS

Who needs to talk — only the server, or both sides? How fast? And what does the network between you and your users allow?

19 **Long polling**
Near-real-time using nothing but ordinary requests.

20 **Server-Sent Events**
A one-way stream over a single HTTP response.

21 **WebSockets**
A persistent, two-way channel.

→ **Shared problem**
Fan-out across many server instances.

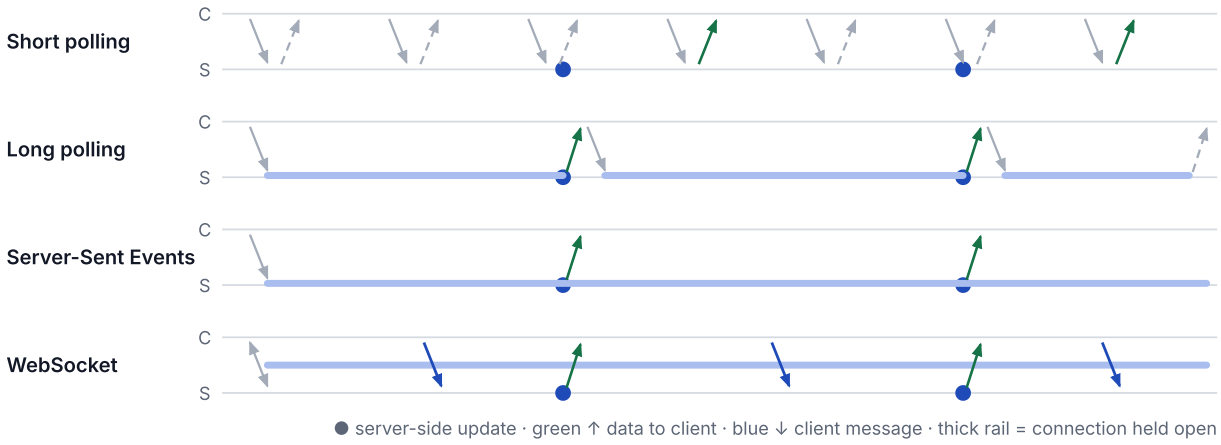


Figure V Four ways to deliver the same two server-side updates. Short polling wastes most requests; long polling holds one open; SSE keeps one response streaming; WebSockets carry traffic both ways.

TECHNIQUE	DIRECTION	TRANSPORT	RESUME AFTER DROP	BEST FOR
Long polling	Server → client	Repeated HTTP requests	Cursor in each request	Fallbacks, infrequent updates, hostile networks
SSE	Server → client	One streaming HTTP response	Built in (`Last-Event-ID`)	Feeds, progress, notifications, token streaming
WebSockets	Both directions	Upgraded TCP connection	You build it	Chat, collaboration, multiplayer, trading UIs

19 Long Polling

Hold the request open until there is news, then immediately ask again.

SOLVES	COSTS	CONSISTENCY
Near-real-time updates where only plain HTTP is reliable	A full request per update; many requests held open	Events can be missed between requests without a cursor

⚠ THE PROBLEM

An order-tracking page is used heavily by enterprise customers whose corporate proxies break WebSocket connections. Short polling every two seconds from 100,000 open tabs costs 50,000 requests per second — almost all of them returning "nothing new".

How it works

The client sends a request such as `GET /updates?since=41`. Instead of answering immediately, the server holds it until there is something new — then responds, and the client instantly sends the next request. If nothing happens for 20–30 seconds, the server returns an empty response and the client reconnects. Keep that hold time below the idle timeouts of every load balancer and proxy in the path, which are often 60 seconds.

The **cursor** is what makes this correct. Between one response and the next request there is a gap; an event in that gap would be lost if the client only asked "anything new?". Asking "anything after event 41?" lets the server answer from a short buffer, so nothing slips through.

Held requests must be cheap. An async server (Node.js, Go, Netty) parks thousands of them with no thread each. And with several server instances, the event may arrive on instance B while the client's request waits on instance A — so instances subscribe to a shared backplane such as Redis pub/sub to wake the right waiters.

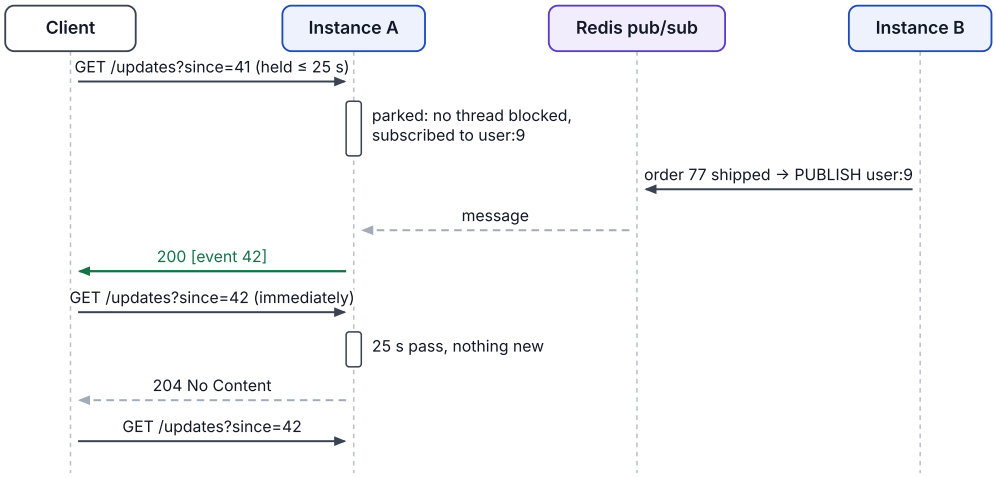


Figure 19.1 Long polling with a cursor. The update arrives on another instance and reaches the waiting request through the backplane; an idle hold ends in a 204 and an immediate re-poll.

Follow-ups to expect

Q Why not plain short polling?

Fine at small scale. At large scale most polls return nothing; long polling removes the empty requests and cuts delivery latency.

Q How long should the server hold a request?

20–30 seconds, below every idle timeout on the path, with jitter so clients don't re-poll in lockstep.

Q How does an instance know which requests to wake?

It keeps waiters in a map keyed by user or topic and subscribes to the matching backplane channels.

Q Does HTTP/2 change the picture?

It makes held requests cheaper by multiplexing them, but for continuous streaming SSE over HTTP/2 is still simpler.

REACH FOR IT WHEN

- Proxies, firewalls or old clients block streaming.
- Updates are infrequent and a second of latency is fine.
- You need a fallback behind SSE or WebSockets.

WATCH OUT FOR

- Per-update overhead: headers, auth, possibly TLS handshakes.
- Hold times longer than load-balancer idle timeouts.
- Every client reconnecting at once after a deploy.

INTERVIEW CUE

"Compare it with SSE and WebSockets on three axes: latency, infrastructure compatibility, and direction of communication. Long polling wins on compatibility and loses on overhead."

20 Server-Sent Events

One long-lived HTTP response that streams events from server to client.

SOLVES	COSTS	CONSISTENCY
Server-to-client push with minimal machinery	One-way only; long-lived connections to plan for	Resumable: the browser reports the last event ID it saw

⚠ THE PROBLEM

A CI dashboard needs live build progress, a notifications bell needs new items as they occur, and an AI assistant must stream its answer token by token. All three are strictly server-to-client; the client's own actions are ordinary HTTP requests.

How it works

The browser opens `new EventSource('/stream')`. The server replies with `Content-Type: text/event-stream` and never finishes the response; it writes small text frames whenever something happens. Commands from the client go over normal requests. It's still just HTTP — it passes through most proxies, uses existing auth cookies, and needs no protocol upgrade.

Resume is built in. Each frame may carry an `id`. If the connection drops, `EventSource` reconnects automatically after the `retry` interval and sends `Last-Event-ID`; the server replays anything newer from a short buffer. Send a comment line every 15–30 seconds as a heartbeat so idle proxies don't cut the stream.

The sharp edges are infrastructural. Proxies that buffer responses delay every event — disable buffering (in nginx, `X-Accel-Buffering: no`) and avoid compressing the stream in a buffering layer. Over HTTP/1.1 browsers allow about six connections per origin across all tabs, so serve SSE over HTTP/2. `EventSource` cannot set custom headers; use cookies, or stream via `fetch` when you need bearer tokens.

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache

retry: 3000

id: 41
event: progress
data: {"build": 812, "step": "test", "pct": 64}

: heartbeat
```

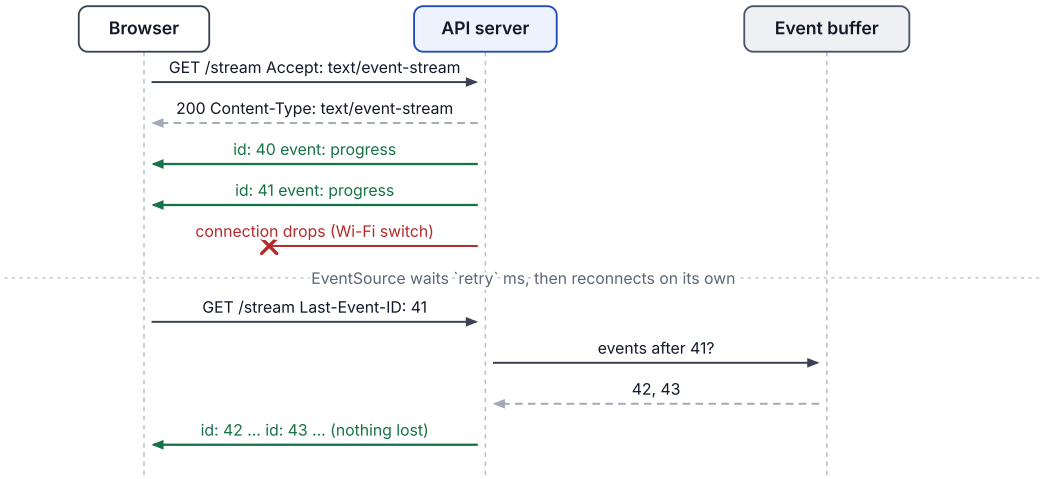


Figure 20.1 An SSE stream survives a network drop. The browser reconnects with the last ID it saw, and the server replays from its buffer.

REACH FOR IT WHEN

- Only the server needs to push: feeds, alerts, progress.
- Streaming LLM output or long computations.
- You want automatic reconnect and resume for free.

WATCH OUT FOR

- Proxy and compression buffering.
- The HTTP/1.1 per-origin connection limit.
- Text only — binary data needs encoding.

INTERVIEW CUE

"If only the server needs to push updates, SSE is the natural choice — simpler than WebSockets, works over plain HTTP, and reconnects and resumes on its own."

21 WebSockets

A persistent, full-duplex channel for when both sides need to talk, constantly.

SOLVES	COSTS	CONSISTENCY
Low-latency, frequent messaging in both directions	Stateful connections that complicate scaling, deploys and auth	No delivery guarantees or resume — the app protocol provides them

⚠ THE PROBLEM

A collaborative document editor must deliver every keystroke and cursor movement between dozens of users in well under 100 ms. Traffic flows constantly in both directions, and per-message HTTP overhead would dwarf the messages themselves.

How it works

The client starts with an HTTP request carrying `Upgrade: websocket`; the server answers `101 Switching Protocols`, and the same TCP connection now carries framed messages in both directions with a few bytes of overhead each. Authenticate during the handshake, and re-check when tokens expire — the connection may outlive them by hours.

Connections are **stateful**, and that is the source of every scaling problem. Alice may be connected to gateway 1 and Carol to gateway 3; a message for their shared document must cross between them. The standard answer is a **pub/sub backplane** (Redis, NATS, Kafka) keyed by room or user, plus a registry of who is connected where. Plan capacity in connections and memory per connection, and raise file-descriptor limits.

WebSockets give you a pipe, not a protocol. You need heartbeats (ping/pong) to detect half-open connections, message IDs and acknowledgements, and resume from a sequence number after reconnect. Reconnect with jittered backoff — a deploy that drops 50,000 sockets at once produces a reconnect storm, so drain gateways gradually. Watch slow clients: if a socket's send buffer keeps growing, drop messages or disconnect rather than letting memory grow.

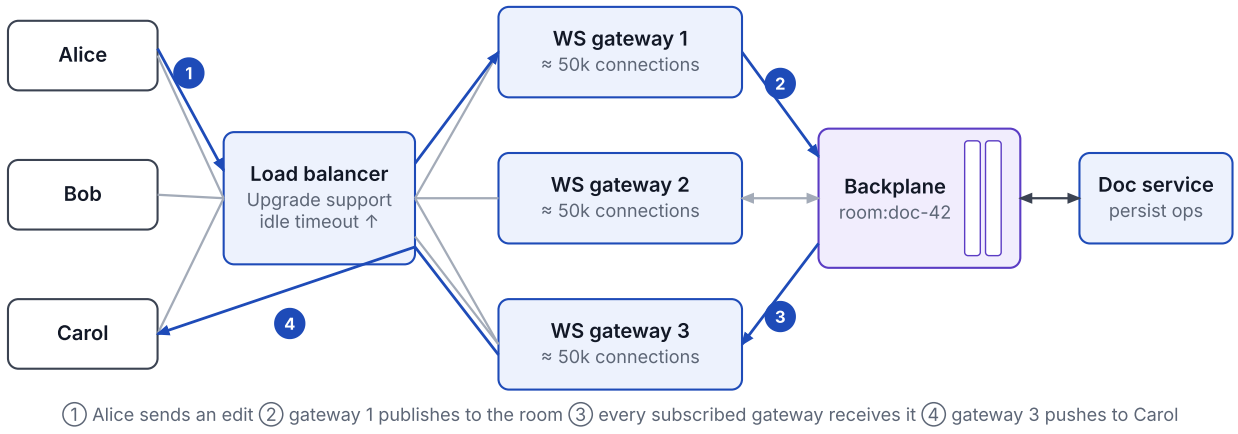


Figure 21.1 Horizontal scale-out. Gateways hold connections; a backplane routes each message to whichever gateways hold the recipients.

Follow-ups to expect

Q How many connections per server?

Tens to hundreds of thousands on a tuned async server. The limits are memory per connection, file descriptors and fan-out CPU — not the protocol.

Q How do you deploy without dropping everyone?

Stop accepting new connections, send a reconnect hint, drain over minutes, and let jittered reconnects spread across the new fleet.

Q Do you need sticky sessions?

Not for WebSockets themselves — a connection stays where it is. Only polling fallbacks, which make many requests, need stickiness.

Q Should messages be persisted?

The socket is ephemeral. Persist operations with sequence numbers so a reconnecting client can catch up from where it left off.

REACH FOR IT WHEN

- Clients send frequent, latency-sensitive messages too.
- Chat, collaboration, multiplayer, live trading.
- Per-message overhead must be minimal.

WATCH OUT FOR

- Reconnect storms after deploys or network blips.
- Load-balancer idle timeouts and upgrade support.
- Backpressure from slow clients; auth expiry mid-session.

INTERVIEW CUE

"I'd choose WebSockets only when the client also needs to send frequent low-latency messages. If it's just server push, SSE is simpler to run."

PART VI OF VI

VI

Putting it together

Knowing twenty-one patterns is not the skill. The skill is hearing a requirement, locating the pressure it creates, choosing the least expensive pattern that relieves it, and saying out loud what that choice costs. This part turns the guide into a method you can run in an interview or a design review.

> THE METHOD IN ONE LINE
>
> **Symptom → cheapest relieving pattern → its cost → its failure behaviour → the next pattern, only if the numbers demand it.**

A **From symptom to pattern**
An escalation matrix for the eight common pressures.

B **The four questions**
Bottleneck, consistency, failure, scale.

C **Worked example**
A flash-sale checkout that uses a dozen patterns.

D **Final cheat sheet**
Every pattern on one page.

> ⓘ A NOTE ON INTERVIEWS
>
> Interviewers rarely reward the most sophisticated architecture. They reward a design that is justified by numbers, honest about its trade-offs, and explicit about what happens when things go wrong. Saying "I'd start without a cache and add one when the read rate passes a few thousand per second" is a senior answer.

From symptom to pattern

Start from what hurts. Move right only when the column to the left has been tried and measured.

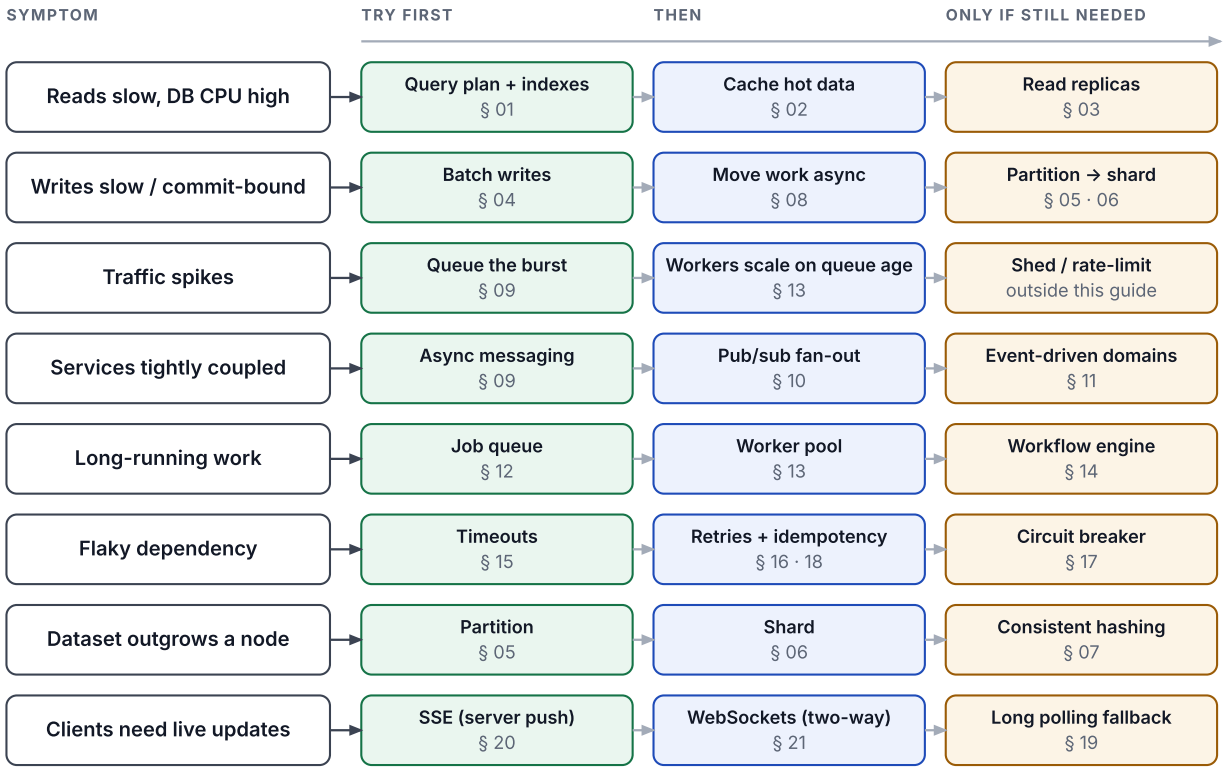


Figure 6.1 Escalation matrix. Green is cheap and reversible; amber is powerful but expensive to operate and hard to undo. Section numbers point to the relevant pattern.

ⓘ WHY LEFT TO RIGHT

Each column to the right adds infrastructure, failure modes and on-call burden. Teams that skip columns usually end up running the expensive system *and* fixing the cheap problem later anyway.

ⓘ WHEN TO SKIP AHEAD

When the arithmetic says so. If projected writes exceed what any single node can handle even after batching, plan the shard key now — retrofitting it onto a live system is far harder.

The four questions

Ask them in this order for every component you draw. Most weak designs skip the third.

1

Where is the bottleneck?

Name the resource that runs out first — CPU, connections, disk, a single primary, a third-party rate limit.

- Reads, writes, or both? What ratio?
- Is it each operation's cost, or the count of operations?
- Is it one hot key or broad load?

2

How stale can it be?

This single answer decides whether caches, replicas, async processing and events are even allowed.

- Which reads must see the user's own writes?
- Which can be a second, a minute, an hour old?
- Where would the UI expose eventual consistency?

3

What happens when it fails?

For every arrow in the diagram, walk through the failure cases explicitly.

- It times out — did it happen or not?
- It runs twice — is anything charged, sent or shipped twice?
- It's down for ten minutes — do we queue, degrade or fail?
- A worker dies after the side effect but before the ack?

4

What do the numbers say?

Estimate before choosing. Then pick the simplest mechanism that covers the peak with headroom.

- Requests per second, average and peak.
- Data size today and in two years.
- Payload sizes and latency targets (p50, p99).

Back-of-envelope reference

OPERATION	ORDER OF MAGNITUDE
Main-memory read	~100 ns
SSD random read	~100 µs
Round trip, same data centre	~0.5 ms
Redis GET over the network	~0.5–1 ms
Indexed primary-key query	~1–5 ms
Cross-continent round trip	~70–150 ms

CONVERSION	RULE OF THUMB
Seconds per day	~86,400 ≈ 10^5
1 M requests / day	~12 req/s average
Peak vs average	Plan for 3–10×
1 KB × 1 M rows	~1 GB (before indexes)
10 M DAU × 20 req/day	~2,300 req/s average
Little's law	concurrency = rate × latency

Figures are rough orders of magnitude for reasoning, not benchmarks. Hardware, network and query shape move them substantially.

Worked example: a flash-sale checkout

10,000 units of a sneaker go on sale at noon. 500,000 people are waiting.

> ⚠ **THE REQUIREMENTS, RESTATED AS PRESSURES**
>
> **Reads** spike to hundreds of thousands per second on one product page. **Writes** spike to tens of thousands of checkout attempts per second for a few minutes. **Correctness**: never sell unit 10,001 and never charge anyone twice. **Failure**: the payment provider will throttle and occasionally fail under this load. **Experience**: buyers must learn quickly whether they got a pair.

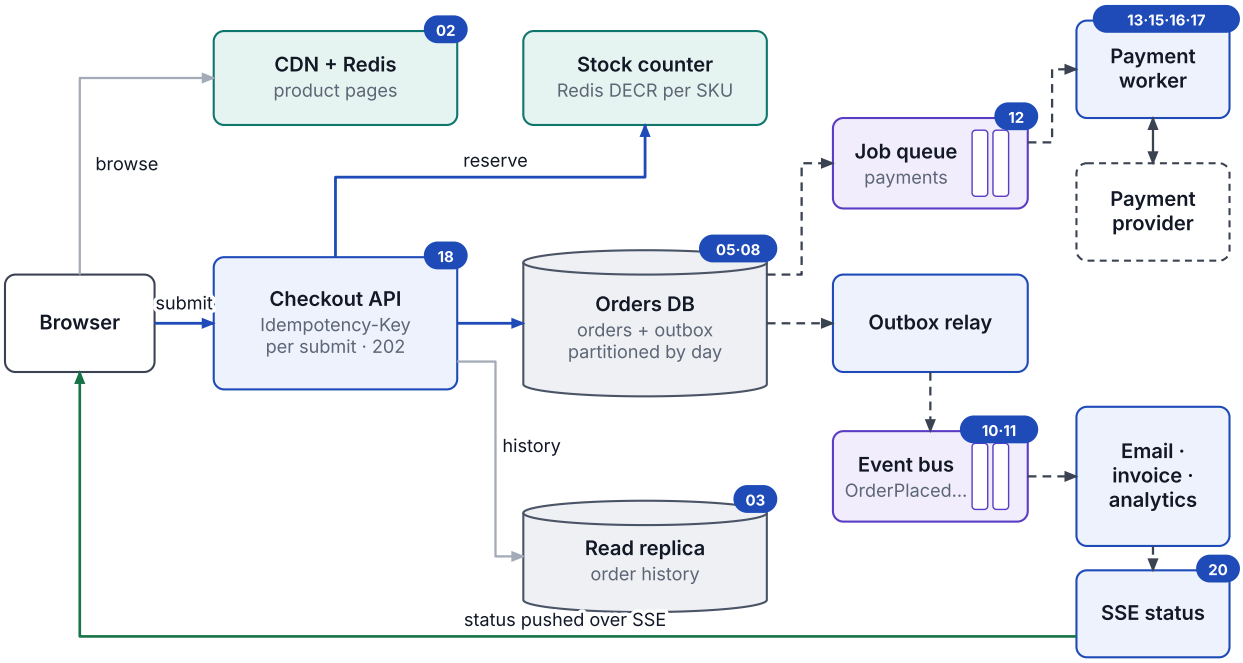


Figure 6.2 The design. Blue labels give the pattern numbers in play at each component. The synchronous path ends at the orders database; everything to the right happens after the response.

1. **Product page from cache.** Static content from the CDN, stock level from Redis with a one-second TTL. A slightly stale "in stock" badge is fine; the reservation step is the real check. 02
2. **Idempotent submit.** The client creates one key per checkout attempt; double-clicks and retries return the same order. 18
3. **Atomic reservation.** A Redis `DECR` on the SKU counter. A result below zero means sold out: undo the decrement and fail fast before touching the database. fast reject
4. **Order plus outbox in one transaction.** The order row and its `OrderPlaced` event commit together, and the API returns `202 Accepted` with an order ID. The table is partitioned by day for easy archiving. 05 · 08
5. **Payment as a job.** Workers drain the payment queue at the rate the provider allows, with per-attempt timeouts, jittered retries and a breaker, sending the order ID as the provider's idempotency key. 12 · 13 · 15–17
6. **Fan-out of facts.** The relay publishes events; email, invoicing and analytics each consume at their own pace. 10 · 11
7. **Live status.** The confirmation page subscribes over SSE and flips to "Confirmed" or "Payment failed" the moment the worker finishes. 20
8. **Failure handling.** If payment ultimately fails, a compensating step releases the reserved unit back to the counter — a small saga. 14

TRADE-OFFS TO STATE OUT LOUD

- "Accepted" is not "paid" — the UI must say so honestly.
- The Redis counter and the database can diverge; a reconciliation job compares them after the sale.
- Order history reads from a replica may lag a few seconds.

WHAT'S DELIBERATELY ABSENT

No sharding: a few minutes of tens of thousands of writes per second, mostly rejected at the Redis counter, leaves a modest number of real orders a well-tuned primary can absorb. No WebSockets: the client never needs to talk back. Every omission is a decision, too.

If the interviewer pushes

Q What if Redis loses the stock counter?

Pause the sale, rebuild the counter from the database (initial stock minus confirmed and pending orders), then reopen.

Q What about bots?

Per-account and per-IP rate limits, a waiting room, and bot challenges at the edge. Outside this guide, but name it.

Q Why not a queue in front of order creation?

A virtual waiting room that admits buyers at a fixed rate also works. It trades latency for a hard cap on load — worth it for much larger sales.

Q How would you test this before noon?

Load-test at twice the expected peak on production-like data, and run a game day that kills the payment provider and a Redis node mid-sale.

Final cheat sheet

Every pattern, what it buys, and the price you should name before anyone asks.

#	PATTERN	SYMPTOM IT ADDRESSES	WHAT IT BUYS	WHAT IT COSTS
01	Indexing	Slow selective queries	Log-time lookups instead of scans	Write overhead, storage, memory
02	Caching	Repeated expensive reads	Sub-millisecond hot reads	Staleness, invalidation, stampedes
03	Read replicas	Read-bound database	Horizontal read capacity	Replication lag, read-your-writes bugs
04	Batching	Many tiny writes	Fewer round trips and commits	Latency floor, partial failures
05	Partitioning	Huge table, natural slices	Pruning, instant retention	Queries without the key scan all
06	Sharding	One node at capacity	Write and storage scale-out	Cross-shard work, resharding
07	Consistent hashing	Changing node set	~1/N keys move on change	Needs virtual nodes; no replication
08	Async processing	Slow request path	Fast, decoupled responses	Eventual consistency, dual writes
09	Message queues	Bursts, uneven rates	Load levelling, one handler per item	Duplicates, ordering limits
10	Pub/Sub	One event, many consumers	Independent fan-out, replay	Lag per subscriber, schemas
11	Event-driven arch.	Chained synchronous calls	Independent services and teams	Invisible flows, harder debugging
12	Job queues	Deferred or scheduled work	Retries, delays, priorities	Duplicate execution, poison jobs
13	Background workers	Heavy work in the API	Isolated, scalable execution	Shutdown, stuck jobs, downstream load
14	Workflow engines	Multi-step, long-running process	Durable state, compensation	New platform, determinism, versioning
15	Timeouts	Hung dependencies	Bounded resource usage	Unknown outcomes, false failures
16	Retries + backoff	Transient failures	Automatic recovery	Retry storms, duplicate effects
17	Circuit breakers	Persistently failing dependency	Fail fast, room to recover	Tuning, may reject good calls
18	Idempotency	Retries and redelivery	Exactly-once effects	Key storage, atomicity care
19	Long polling	Live data on plain HTTP	Maximum compatibility	Request per update, held requests
20	SSE	Server-to-client push	Simple streaming, built-in resume	One-way, proxy buffering
21	WebSockets	Two-way real-time	Lowest-latency full duplex	Stateful scaling, reconnect storms

Every pattern in this guide is a trade. The engineers worth learning from are not the ones who know the most patterns — they're the ones who can tell you, precisely, what each one will cost them, and why it's worth paying here.